

Introduction to Data Science

Unit-VI

Basic Machine Learning



SREENIDHI
EDUCATIONAL GROUP

SREENIDHI
INSTITUTE OF
SCIENCE AND
TECHNOLOGY



Syllabus

- Basic Machine Learning:
- Introduction to machine learning
- Types of machine learning:
- Supervised and unsupervised
- Types of Supervised learning: Classification and regression
- Classifiers – Decision tree and k-NearestNeighbors (k-NN)
- Types of unsupervised: Clustering k-means Association Rule mining

Things Focus in this Chapter...

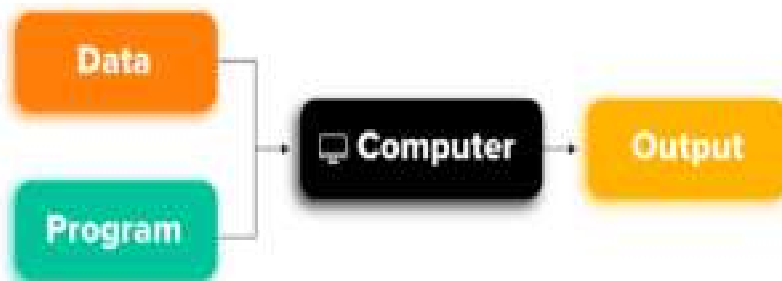
- Defining the different types of machine learning, along with examples of each kind.
- Areas in regression, classification, and more What is machine learning and how it is used in data science.
- The differences between machine learning and statistical modeling and how machine learning is a broader category of the latter.

What is Machine Learning?

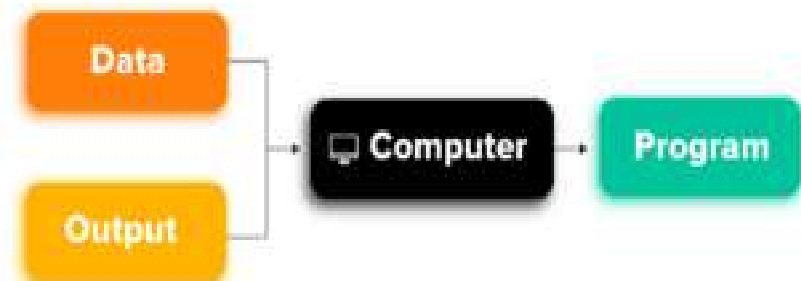
- Machine learning as giving computers the ability to learn from data without being given explicit *rules* by a programmer.
- This definition still holds true. Machine learning is concerned with the ability to ascertain certain patterns (signals) out of data, even if the data has inherent errors in it (noise).
- Machine learning models are able to learn from data without the explicit help of a human. That is the main difference between *machine learning models* and *classical algorithms*.

- **Classical algorithms** are told how to find the best answer in a complex system and the algorithm then searches for these *best solutions* and often works faster and more efficiently than a human.
- In **machine learning**, the model is not told the best solution and instead, is given several examples of the problem and is told, figure out the best solution.

TRADITIONAL PROGRAMMING



MACHINE LEARNING



- Machine learning is **just another tool** in the tool belt of a data scientist. It is on the same level as statistical tests (chi-square or t-tests) or uses base probability/statistics to estimate population parameters.
- Machine learning is a **game of correlations and relationships**.
- Most machine learning algorithms in existence are concerned with finding and/or exploiting **relationships between datasets** (often represented as columns in a Pandas Data-frame).
- *Once machine learning algorithms can pinpoint on certain correlations, the model can either use these relationships to predict future observations or generalize the data to reveal interesting patterns.*

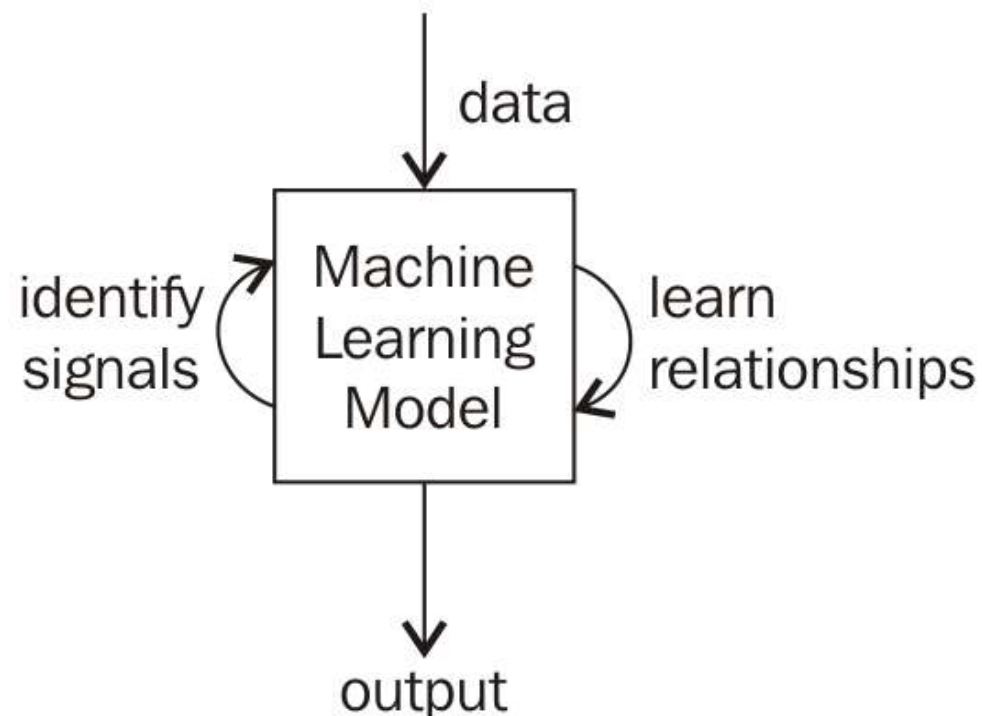
Assumptions for any ML algorithm

- The data used is, for the most part, is preprocessed and cleaned.
- Almost *no machine learning model will tolerate dirty data* with missing values or categorical values. Use dummy variables and filling/dropping techniques to handle these discrepancies.
- Each row of a cleaned dataset represents a single observation of the environment we are trying to model.
- If our *goal is to find relationships between variables*, then there is an assumption that there is some kind of relationship between these variables.
- *These models are not able to communicate that there might not be a relationship.*

- Machine learning models are generally considered semiautomatic, which means that intelligent decisions by humans are still needed.
- The machine is very smart but has a hard time putting things into context.
- Most machine learning models are sensitive to noisy data.
- This means that the models get confused when you include data that doesn't make sense.
- *They are all too important and often ignored by novice data scientists*

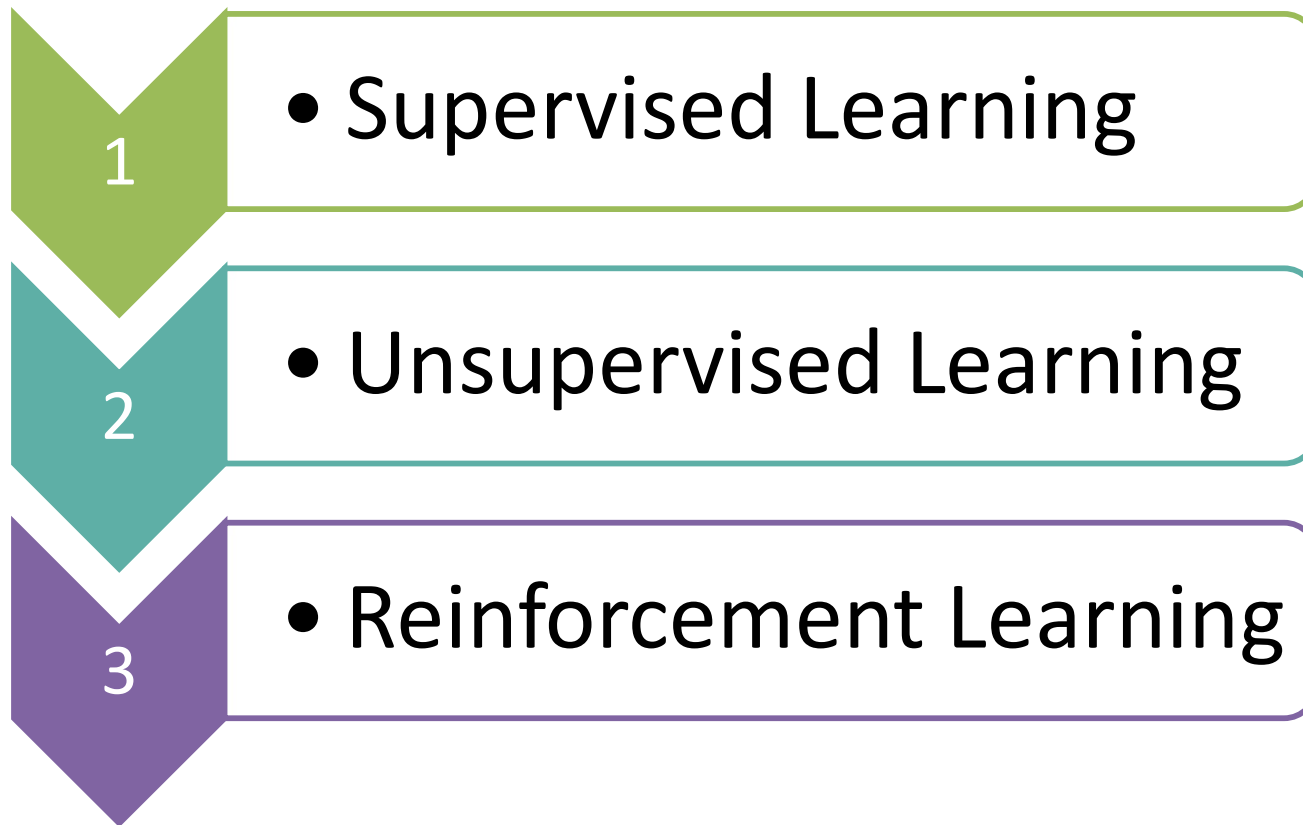
How does machine learning work?

- However, in general, machine learning works by taking in data, finding relationships within the data, and giving as output what the model learned, as illustrated in the following diagram:



Types of machine learning

- Branching off of the top level of machine learning, there are the following three subsets:



1. Supervised Learning

- Supervised learning, as the name suggests, involves **a guided learning process.**
- The core idea behind supervised learning is to **train a model using labeled data**, where each data point consists of input features and a corresponding output or label.
- Supervised learning **finds associations** between features of a dataset and a target variable.
- These associations **allow supervised models to make predictions based on past examples.**

- For example, supervised learning models might try to find the association between **a person's health features** (heart rate, obesity level, and so on) and that person's risk of having a heart attack (the target variable).
- Supervised machine learning models are often called **predictive analytics models**, named for their ability to predict the future based on the past.
- Supervised machine learning requires a certain type of data called **labeled data**.
- This means that we must teach our model by giving it **historical examples** that are labeled with the correct answer.

- The predictors, which are the columns that will be used to make our prediction.
- These are sometimes called features, inputs, variables, and independent variables.
- The response, which is the column that we wish to predict. This is sometimes called outcome, label, target, and dependent variable.

Supervised Learning

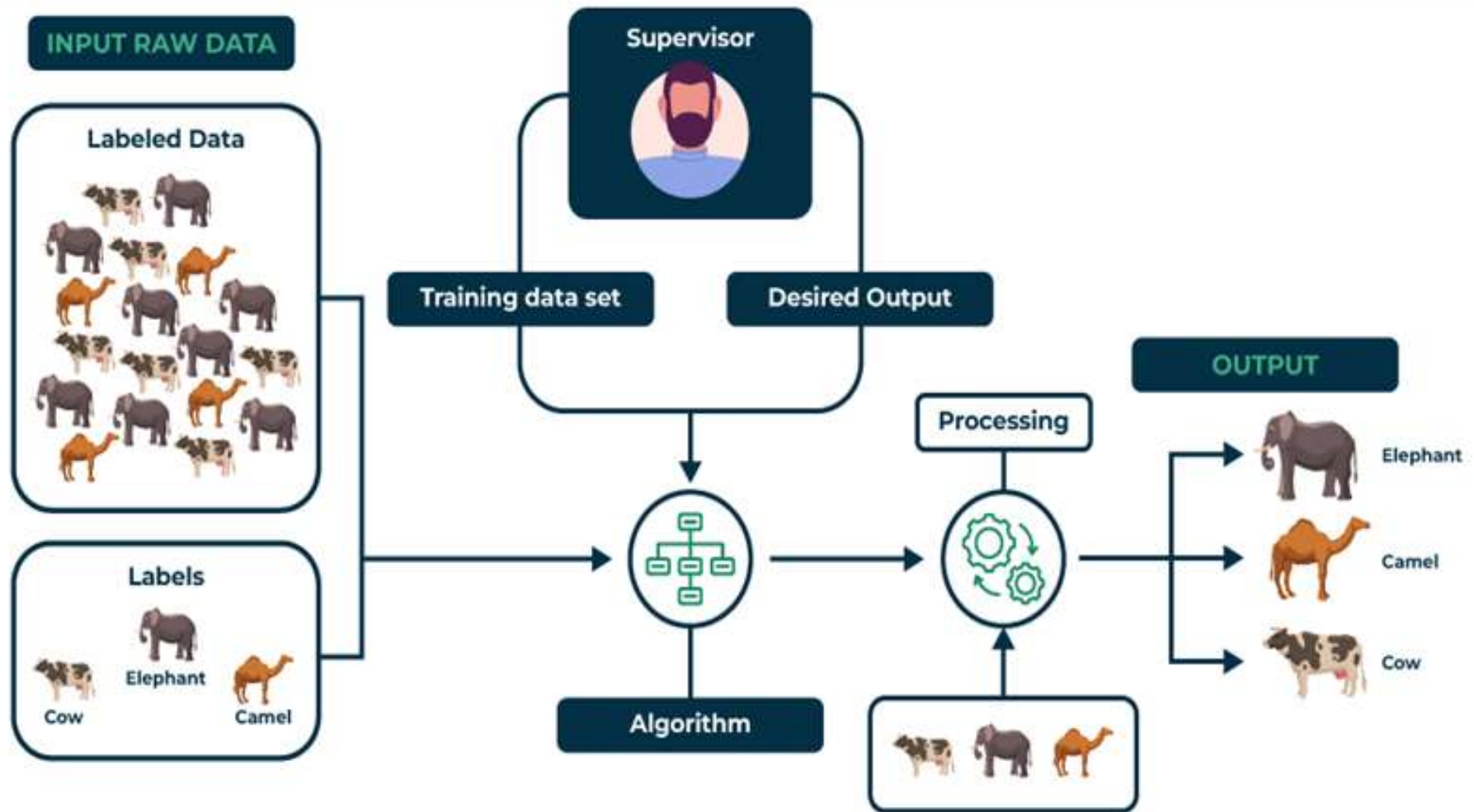


Fig. Principle of Supervised Learning

Example – heart attack prediction

- Suppose we wish to *predict* if someone will have a heart attack within a year.
- To predict this, we are given that person's cholesterol, blood pressure, height, their smoking habits, and perhaps more.
- From this data, we must ascertain the likelihood of a heart attack.
- This is a supervised machine learning problem because
 - Making a prediction about someone
 - Using historical training data to find relationships between medical variables and heart attacks.

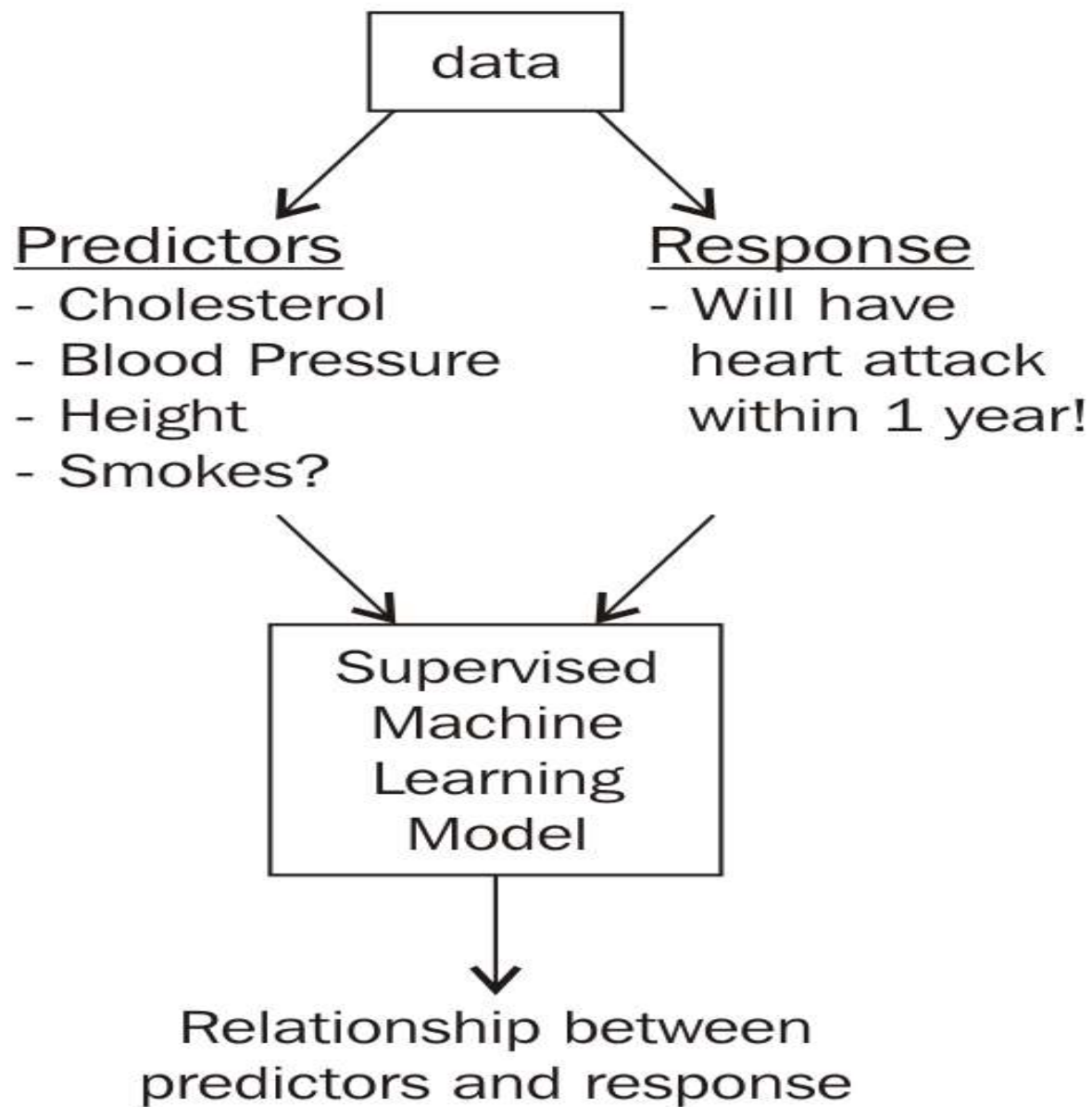


Fig. Supervised Learning on Heart Attack Prediction

- One of the *biggest drawbacks* of supervised machine learning is that we need this *labeled data*, which can be *very difficult* to get a hold of.
- Suppose we wish to *predict heart attacks*, we might need thousands of patients along with all of their filled in *medical information and years' worth of follow-up records* for each person, which could be a nightmare to obtain.
- *In short, supervised models use historical labeled data in order to make predictions about the future.*
- Some possible applications for supervised learning include:
 - *Stock price predictions*
 - *Weather predictions*
 - *Crime predictions*

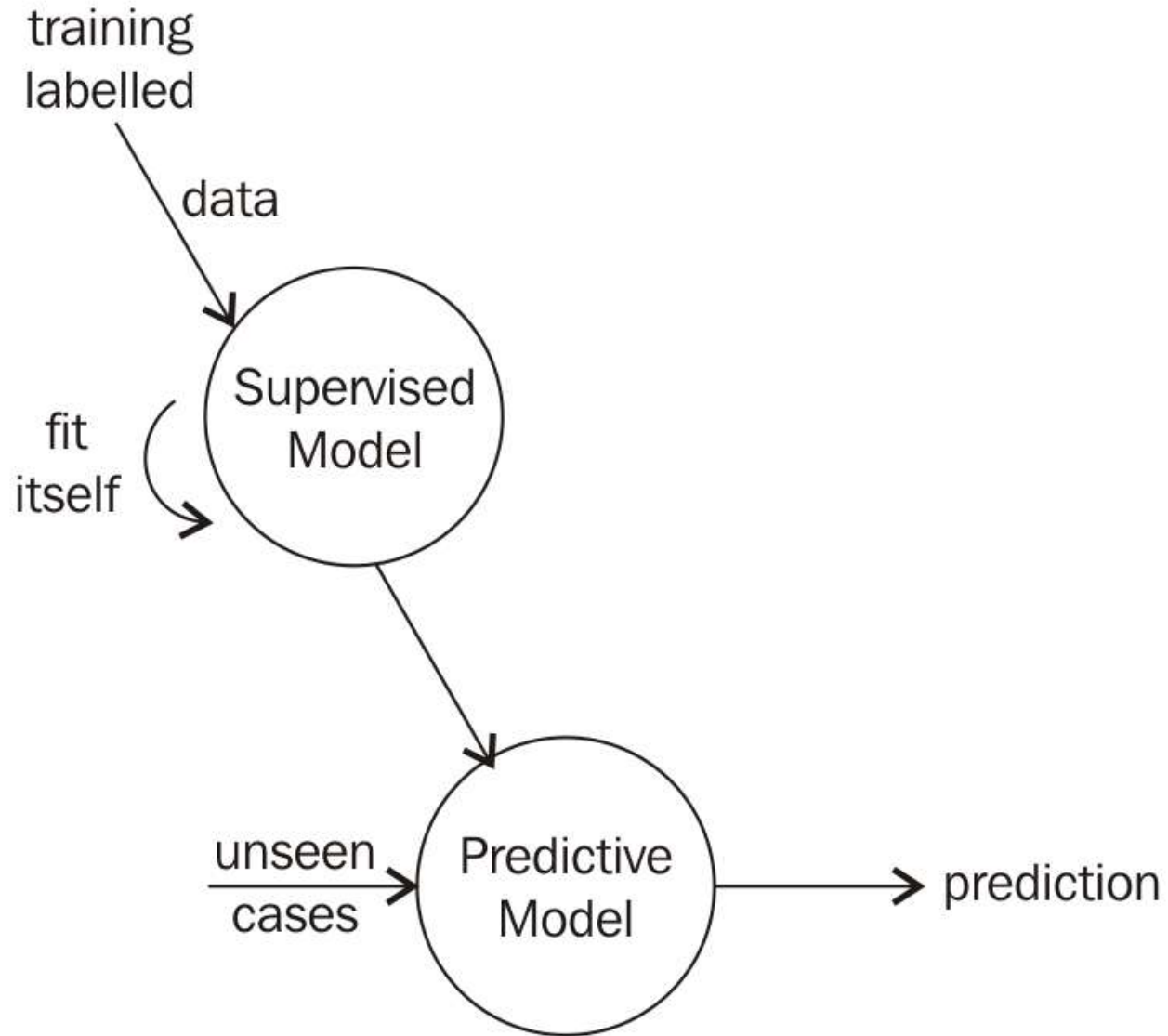


Fig. Methodology of Supervised Learning

It's not only about predictions

- Supervised learning exploits the relationship between the predictors and the response to make predictions, **but sometimes, it is enough just knowing that there even is a relationship.**
- Suppose we are using a supervised machine learning model to predict whether or not a customer will purchase a given item.
- A possible dataset might look as follows:

Person ID	Age	Gender	Employed?	Bought the product?
1	63	F	N	Y
2	24	M	Y	N

- This speaks to supervised machine learning's ability to *understand which predictors affect the response and how.*
- For example, are women more likely to buy the product, which age groups are prone to decline the product, is there a combination of age and gender that is a better predictor than any one column on its own?
- *As someone's age increases, do their chances of buying the product go up, down, or stay the same?*

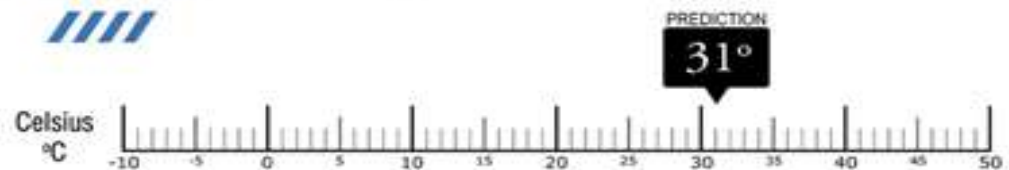
Types of Supervised Learning

- There are, in general, two types of supervised learning models: **regression** and **classification**.
- The difference between the two is quite simple and lies in the **response variable**.



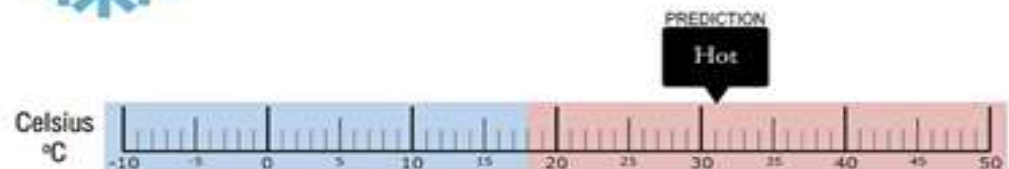
Regression

What temperature will there be tomorrow?



Classification

Will tomorrow be a cold or hot day?



Regression

- Regression models attempt to predict a **continuous** response. This means that the response can take on a **range of infinite values**.
- Consider the following examples:
 - Dollar amounts
 - °° Salary
 - °° Budget
 - Temperature
 - Time
 - °° Generally recorded in seconds or minutes

Classification

- **Classification** attempts to predict a categorical response, which means that the response only has a finite amount of choices.

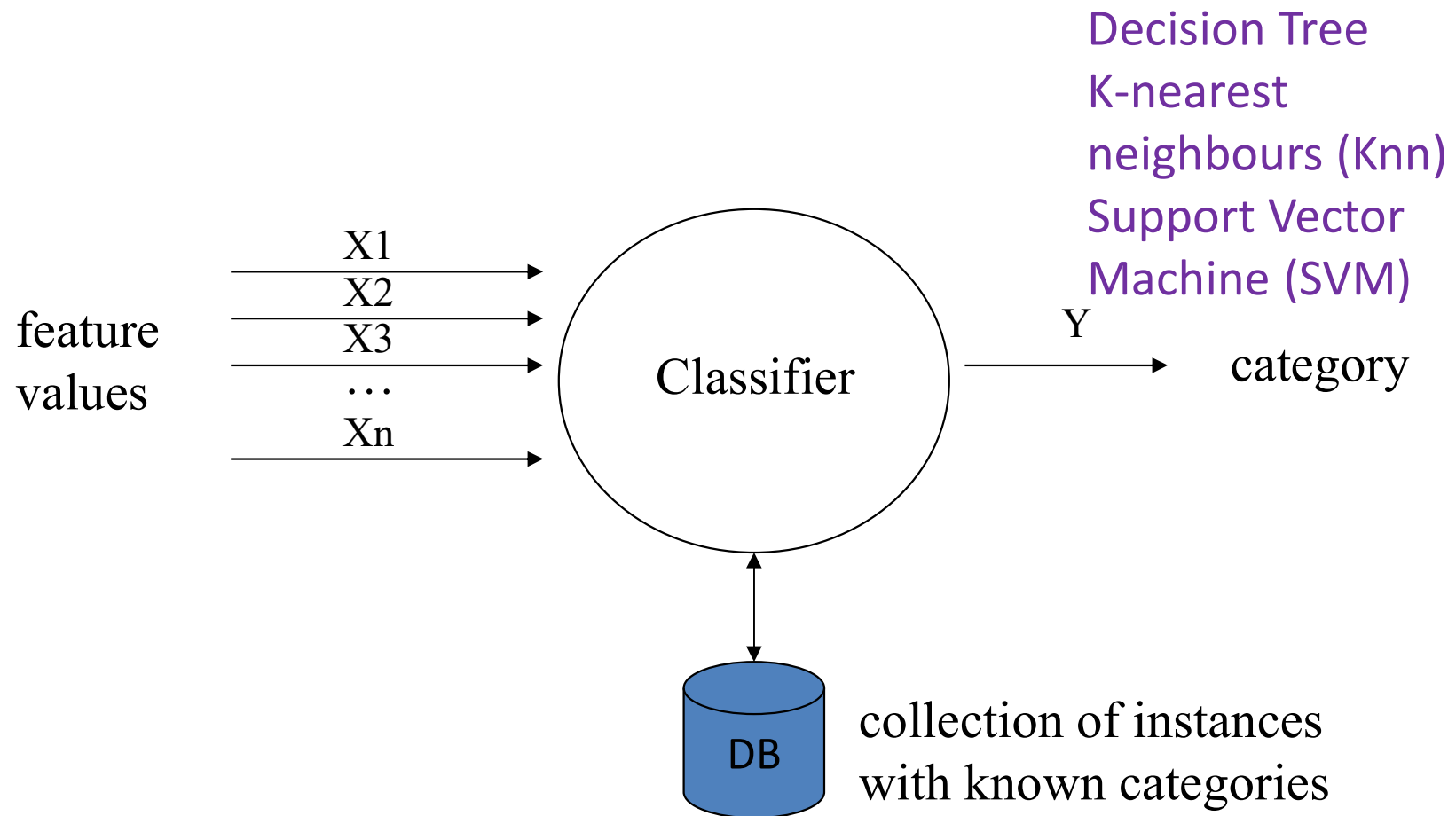
Examples include the ones given as follows:

- **Cancer grade** (1, 2, 3, 4, 5)
- **True/False** questions, such as the following examples:

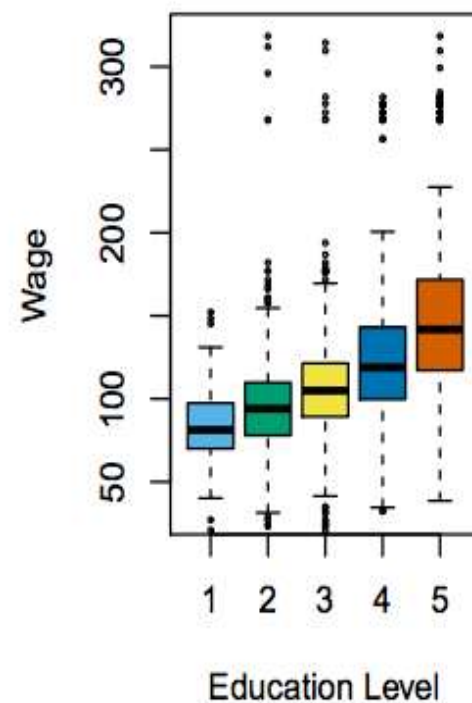
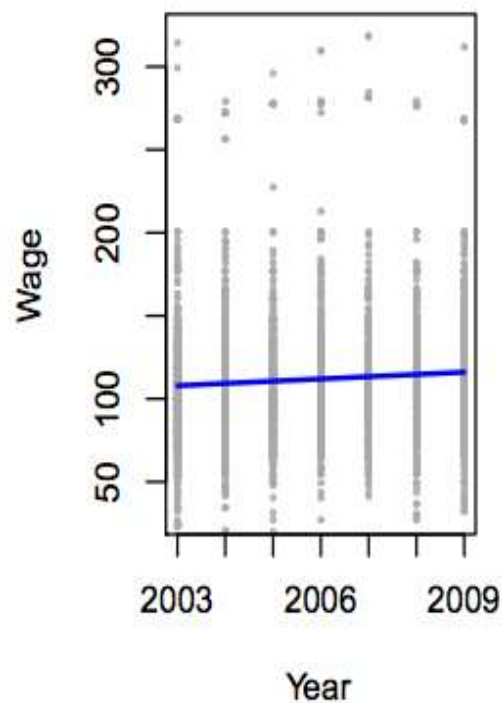
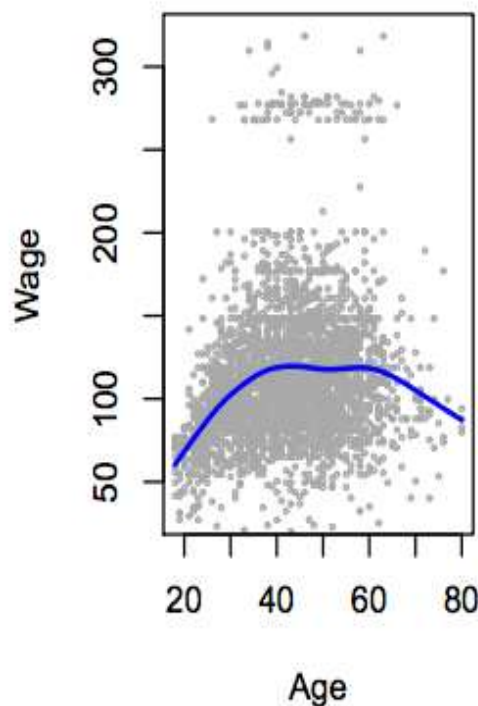
°° "Will this person have a heart attack within a year?"

°° "Will you get this job?"

Classifiers



- Given a photo of a face, who does this face belong to? (facial recognition) **Predict the year someone was born:**
 - °° Note that there are many possible answers (over 100) but still finitely many more
- Example – regression**



Classification

Logistic
Regression

Decision Tree, K
Nearest
Neighbor (Knn),
Support Vector
Machine (SVM)

Regression

Linear

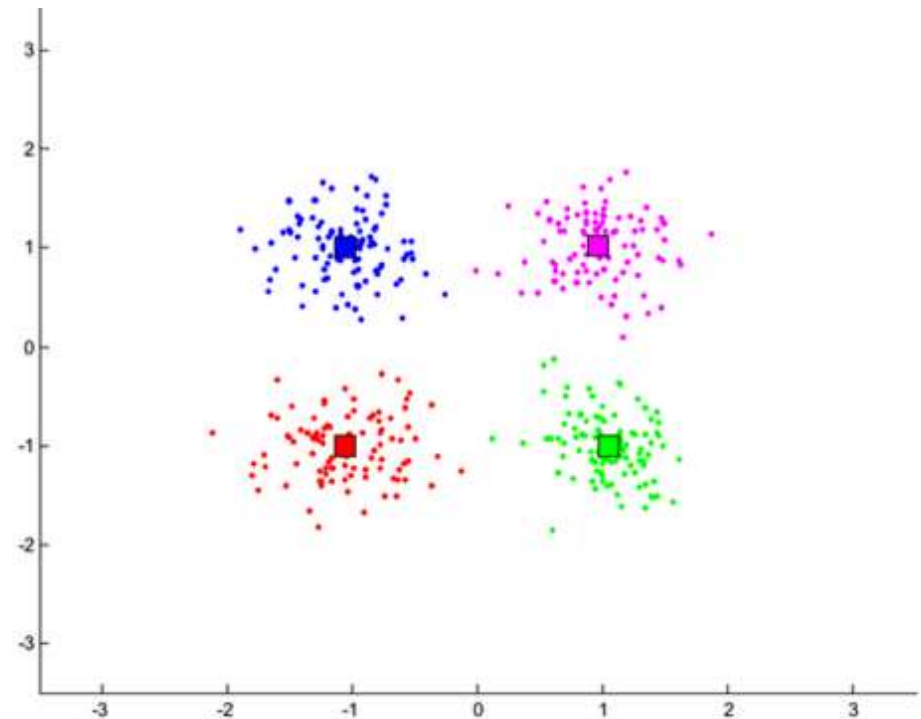
Multi Variable

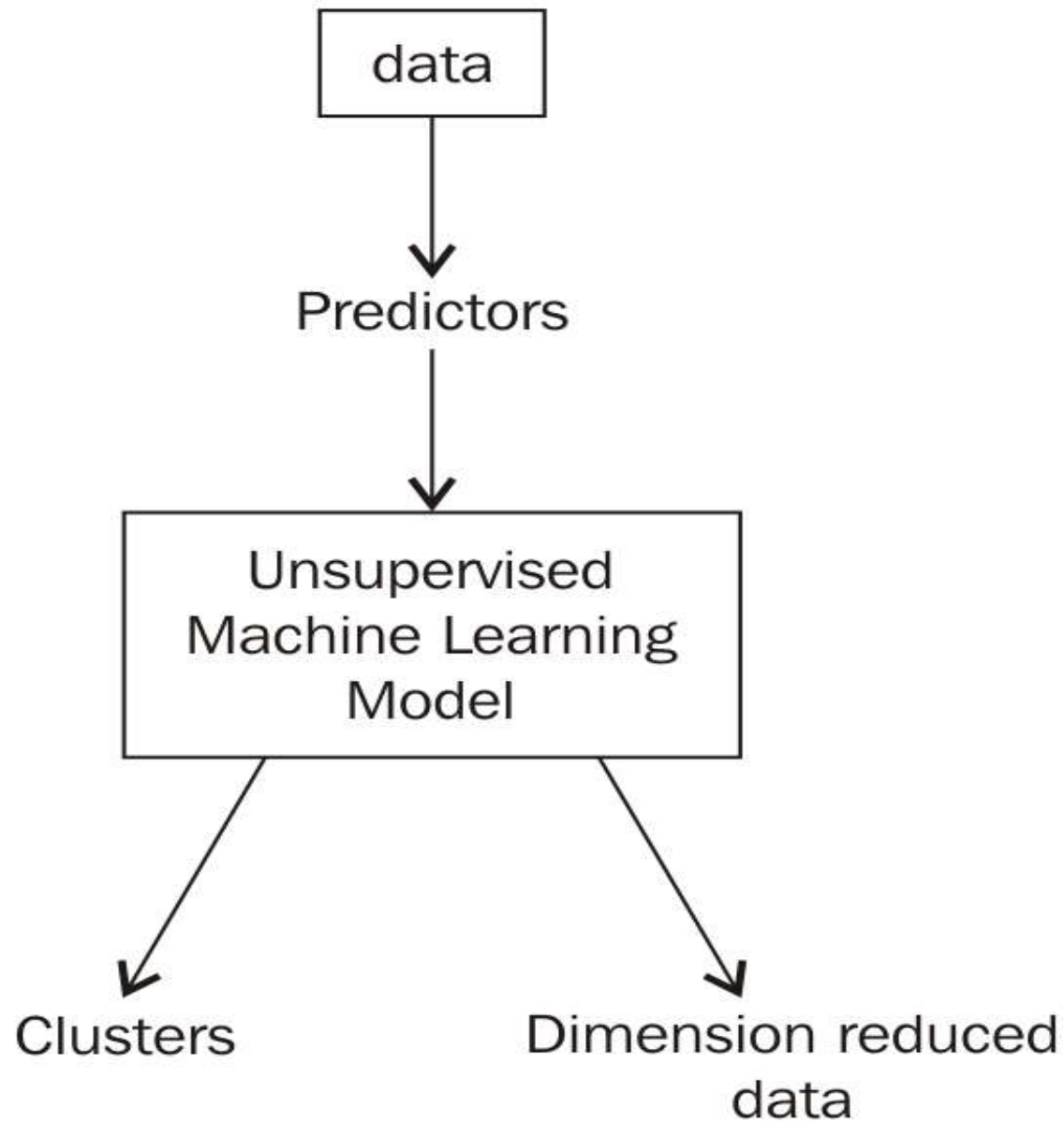
2. Unsupervised learning

- The **second type** of machine learning *does not deal* with predictions but has a much more open objective.
- Unsupervised learning takes in a set of predictors and utilizes relationships between the predictors in order to accomplish tasks, such as the following:
 - **Reducing the dimension** of the data by condensing variables together. (Compression)
 - **Finding groups of observations** that behave similarly and grouping them together.
- The first element on this list is called **dimension reduction** and the second is called **clustering**.

- Both of these are examples of unsupervised learning because they **do not attempt to find a relationship** between predictors and a specific response and therefore are not used to make predictions of any kind.
- Unsupervised models, instead, are utilized to **find organizations and representations** of the data that were previously unknown.

The preceding screenshot is a representation of a cluster analysis. The model will recognize that each uniquely colored cluster of observations is similar to another but different from the other clusters.





Flow of Unsupervised Machine Learning.

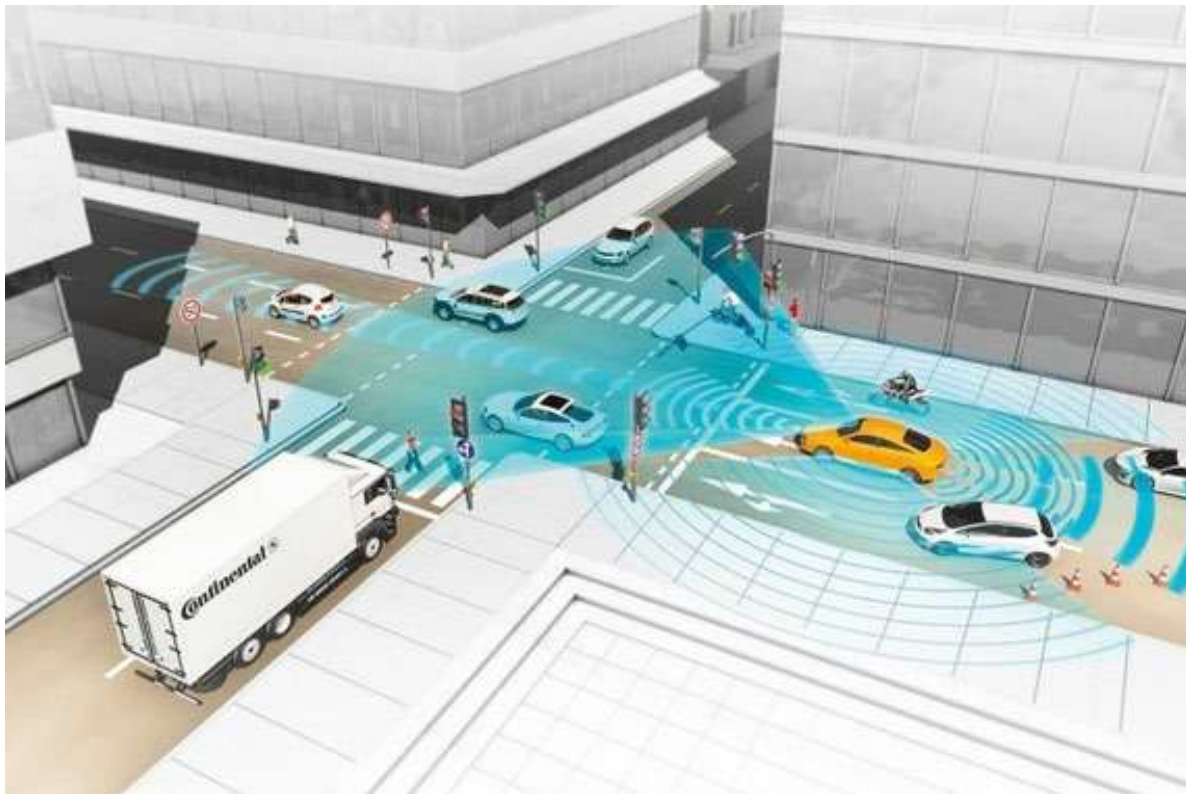
- A big advantage for unsupervised learning is that it does **not require labeled data**, which means that it is much easier to get data that complies with unsupervised learning models.
- Of course, a drawback to this is that we **lose all predictive power** because the response variable holds the information to make predictions and without it our model will be hopeless in making any sort of predictions.
- A big drawback is that it is difficult to see how well we are doing.
- In a regression or classification problem, we can easily tell how well our models are predicting by comparing our models' answers to the actual answers.

- For example, if our supervised model predicts rain and it is sunny outside, the model was incorrect. If our supervised model predicts the price will go up by 1 dollar and it goes up by 99 cents, our model was very close!
- In *supervised modeling*, this concept is foreign because we have no answer to compare our models to.
- *Unsupervised models* are merely suggesting differences and similarities, which then requires a human's interpretation.

3. Reinforcement learning

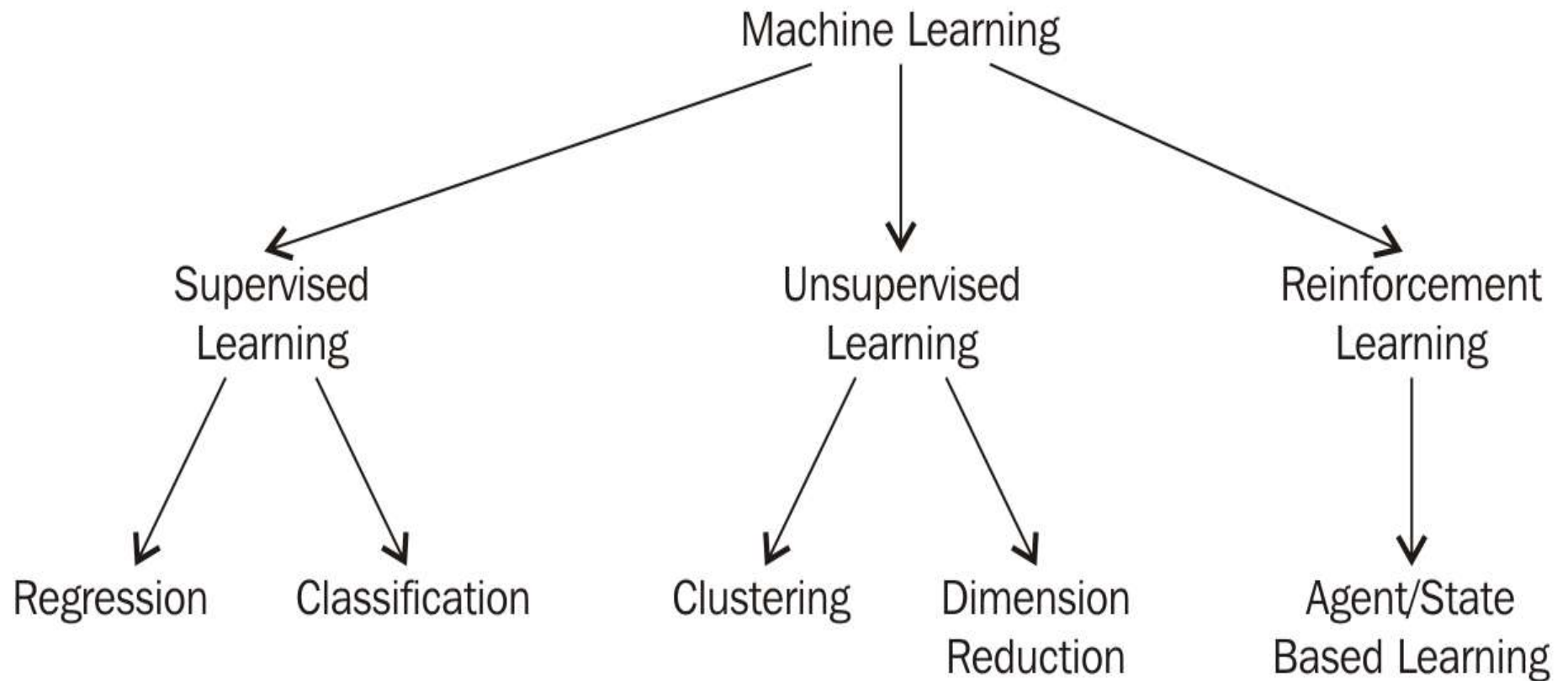
- In reinforcement learning, algorithms get to choose an action in an environment and then are rewarded (positively or negatively) for choosing this action. The algorithm then *adjusts itself and modifies its strategy* in order to accomplish some goal, which is usually to get more rewards.
- This type of machine learning is very popular in AI-assisted game play as *agents* (the AI) are allowed to explore a virtual world and collect rewards and learn the best navigation techniques.
- *This model is also popular in robotics, especially in the field of self-automated machinery, including cars:*

- It can be thought that reinforcement is **similar to supervised learning** in that the agent is learning from its past actions to make better moves in the future;
- however, the **main difference** lies in the reward. The reward does not have to be tied in any way to a "correct" or "incorrect" decision. The reward simply encourages (or discourages) different actions.



Self-driving cars read in sensor input, act accordingly and are then rewarded for taking a certain action. The car then adjusts its behavior to collect more rewards.

Overview of the types of machine learning



Supervised machine learning:

- This exploits relationships between predictors and response variables to make predictions of future data observations.
- Pros:
 - It can make future predictions
 - It can quantify relationships between predictors and response variables
 - It can show us how variables affect each other and how much
- Cons:
 - It requires labeled data (which can be difficult to get)

Unsupervised machine learning:

- This finds similarities and differences between data points.
- Pros:
 - It can find groups of data points that behave similarly that a human would never have noted
 - It can be a preprocessing step for supervised learning.
 - It can use unlabeled data, which is much easier to find
- Cons:
 - It has zero predictive power
 - It can be hard to determine if we are on the right track
 - It relies much more on human interpretation

Reinforcement learning:

- This is reward-based learning that encourages agents to take particular actions in their environments.
- Pros:
 - Very complicated rewards systems create very complicated AI systems
 - It can learn in almost any environment, including our own Earth
- Cons:
 - The agent is erratic at first and makes many terrible choices before realizing that these choices have negative rewards

For example, a car might crash into a wall and not know that that is not okay until the environment negatively rewards it

- °° It can take a while before the agent avoids decisions altogether

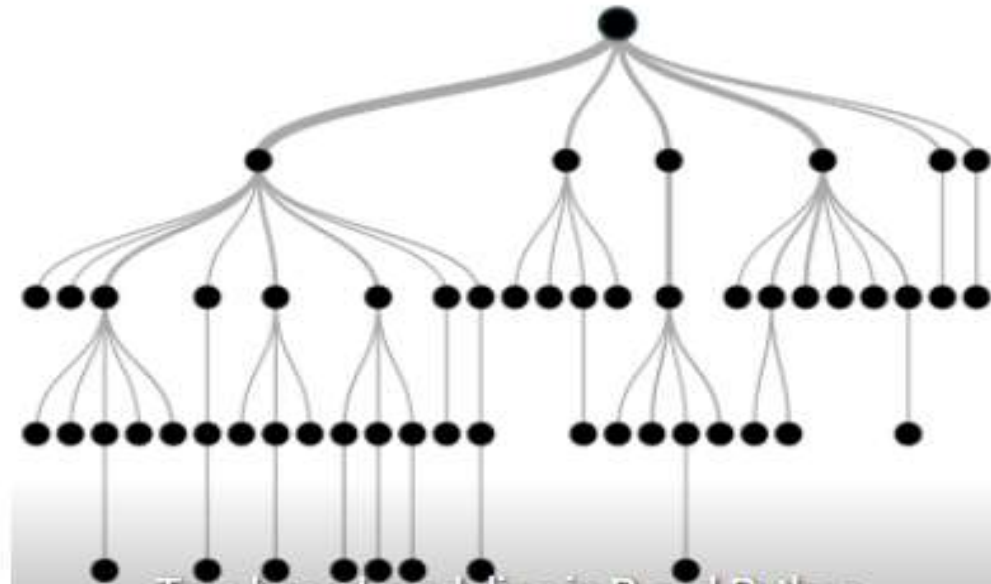
- °° The agent might play it safe and only choose one action and be "too afraid" to try anything else for fear of being punished

Tree Based Algorithms

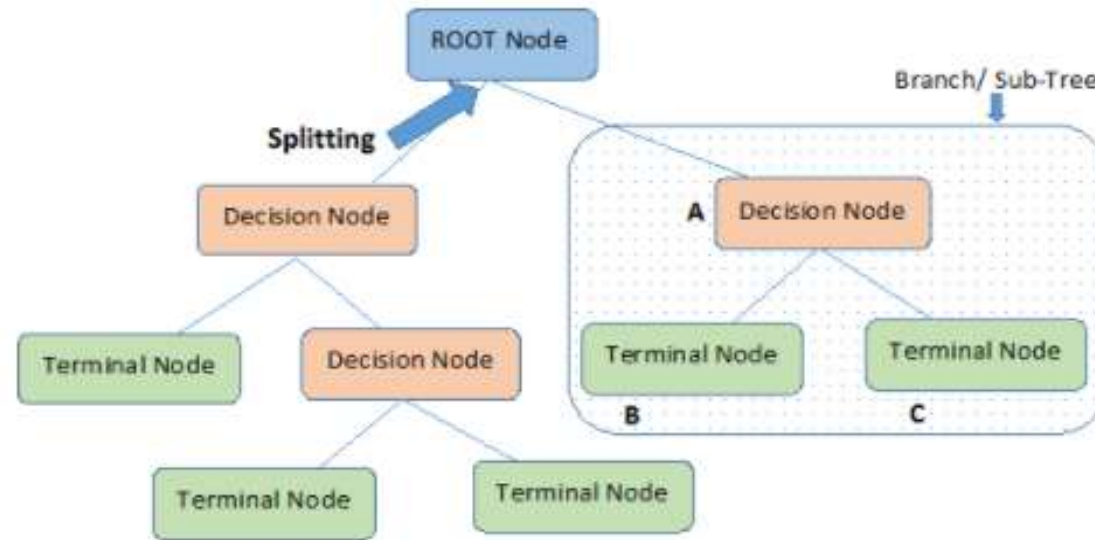
- Decision tree
- Random forest
- Tree based algorithms are considered to be one of the best and mostly used supervised learning methods.
- Tree based algorithms empower predictive models with high accuracy, stability and ease of interpretation.
- Unlike linear models, they map non-linear relationships quite well.
- They are adaptable at solving any kind of problem at hand (classification or regression)

Decision Tree ?

- Decision tree is a type of supervised learning algorithm (having a predefined target variable) that is mostly used in classification problems.
- It works for both categorical and continuous input and output variables.
- In this technique, we split the population or sample into two or more homogeneous sets (or sub-populations).



Basic terminology used with Decision trees:



Note:- A is parent node of B and C.

1. **Root Node:** It represents entire population or sample and this further gets divided into two or more homogeneous sets.
2. **Splitting:** It is a process of dividing a node into two or more sub-nodes.
3. **Decision Node:** When a sub-node splits into further sub-nodes, then it is called decision node.
4. **Leaf/ Terminal Node:** Nodes do not split is called Leaf or Terminal node.
5. **Pruning:** When we remove sub-nodes of a decision node, this process is called pruning. You can say opposite process of splitting.
6. **Branch / Sub-Tree:** A sub section of entire tree is called branch or sub-tree.
7. **Parent and Child Node:** A node, which is divided into sub-nodes is called parent node of sub-nodes where as sub-nodes are the child of parent node.

How does a computer build a regression tree?

Modern decision tree algorithms tend to use a **recursive binary splitting approach**:

1. The process begins at the top of the tree.
2. For every feature, it will examine every possible split, and choose the feature and split such that the resulting tree has the lowest possible mean squared error (MSE). The algorithm makes that split.
3. It will then examine the two resulting regions, and again make a single split (in one of the regions) to minimize the MSE.
4. Keep repeating step 3 until a stopping criterion is met:
 - Maximum tree depth (maximum number of splits required to arrive at a leaf)
 - **Minimum number of observations in a leaf (final) node**

How does the Decision Tree algorithm Work?

- **Step-1:** Begin the tree with the root node, says S , which contains the complete dataset.
- **Step-2:** Find the best attribute in the dataset using **Attribute Selection Measure (ASM)**.
- **Step-3:** Divide the S into subsets that contains possible values for the best attributes.
- **Step-4:** Generate the decision tree node, which contains the best attribute.
- **Step-5:** Recursively make new decision trees using the subsets of the dataset created in step -3. Continue this process until a stage is reached where you cannot further classify the nodes and called the final node as a leaf node.

Attribute Selection Measures

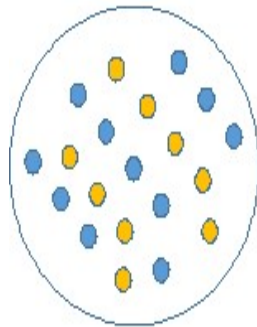
the main issue arises that how to select the best attribute for the root node and for sub-nodes. So, to solve such problems there is a technique which is called as **Attribute selection measure or ASM**

two popular techniques for ASM.

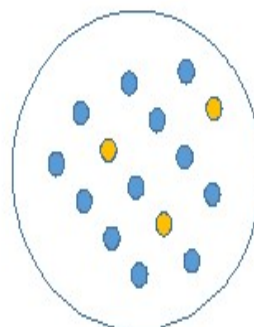
- **a) Entropy** helps us to build an appropriate decision tree for selecting the best splitter.
- Entropy can be defined as a measure of the purity of the sub split. Entropy always lies between 0 to 1

b) Information Gain:

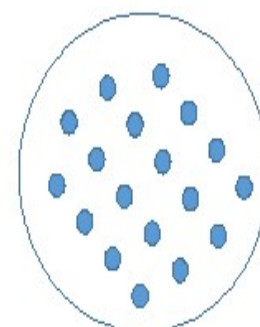
- Look at the image below and think which node can be described easily.
- I am sure, your answer is C because it requires less information as all values are similar.
- On the other hand, B requires more information to describe it and A requires the maximum information. In other words, we can say that C is a Pure node, B is less Impure and A is more impure



A



B



C

- Now, we can build a conclusion that less impure node requires less information to describe it.
- More impure node requires more information.
- Information theory is a measure to define this degree of disorganization in a system known as Entropy.
- If the sample is completely homogeneous, then the entropy is zero and if the sample is an equally divided (50% – 50%), it has entropy of one.

ID3 Decision Tree Example

PlayTennis: training examples

Day	Outlook	Temperature	Humidity	Wind	PlayTennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

Step 1: Entropy of Full Dataset

- Total samples = 14
- Yes = 9, No = 5
- $\text{Entropy}(S) = -9/14 * \log_2(9/14) - 5/14 * \log_2(5/14)$
- $\text{Entropy}(S) = 0.940$

Step 2: Information Gain - Outlook

- Outlook splits:
- Sunny: 5 samples $\rightarrow$ 2 Yes, 3 No
- Overcast: 4 samples $\rightarrow$ All Yes
- Rain: 5 samples $\rightarrow$ 3 Yes, 2 No

- $\text{Entropy}(\text{Sunny}) = 0.971$
- $\text{Entropy}(\text{Overcast}) = 0$
- $\text{Entropy}(\text{Rain}) = 0.971$

- $\text{Gain}(S, \text{Outlook}) = 0.940 - (5/14)*0.971 - (4/14)*0 - (5/14)*0.971$
- $\text{Gain}(S, \text{Outlook}) = 0.247$

Step 3: Information Gain - Humidity

Humidity splits:

- High: 7 samples $\rightarrow$ 3 Yes, 4 No
- Normal: 7 samples $\rightarrow$ 6 Yes, 1 No
- $\text{Entropy}(\text{High}) = 0.985$
- $\text{Entropy}(\text{Normal}) = 0.591$
- $\text{Gain}(S, \text{Humidity}) = 0.940 - (7/14)*0.985 - (7/14)*0.591$
- $\text{Gain}(S, \text{Humidity}) = 0.151$

Iteration 1: Information Gain Summary

- $\text{Gain}(S, \text{Outlook}) = 0.247$ (Highest)
- $\text{Gain}(S, \text{Humidity}) = 0.151$
- $\text{Gain}(S, \text{Wind}) = 0.048$
- $\text{Gain}(S, \text{Temperature}) = 0.029$
- $\rightarrow \text{Root Node} = \text{Outlook}$

Branching Based on Outlook

Sunny → Split by Humidity

- High → No
- Normal → Yes

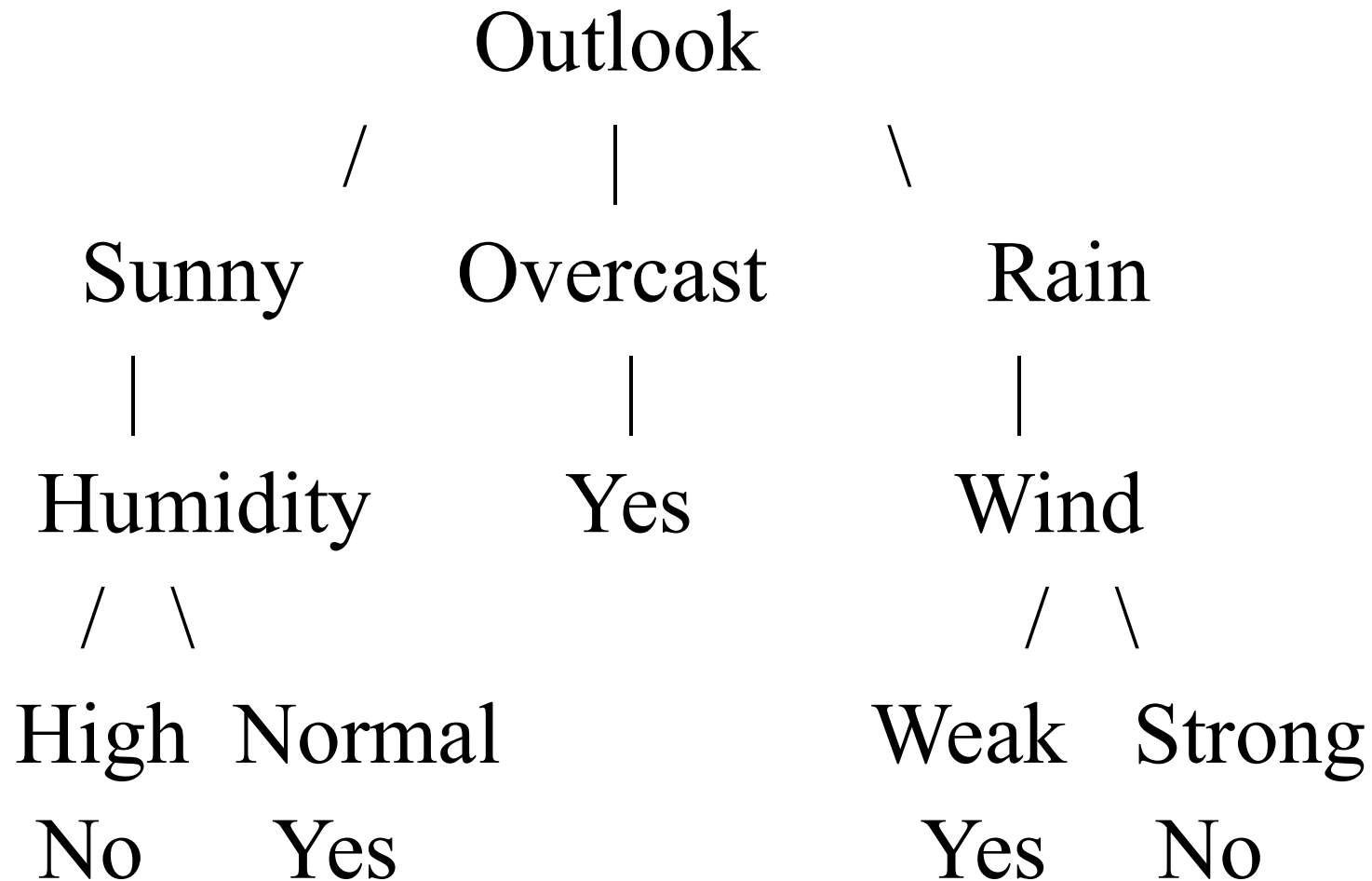
Overcast → Yes

Rain → Split by Wind

- Weak → Yes
- Strong → No

Note: Separate calculations to be done for Sunny and Rain
(Entropy, IG)

Final Decision Tree



Decision Tree: Classification Vs Regression

The following table briefly summarizes the differences between classification and regression decision trees:

Regression trees	Classification trees
Predict a quantitative response	Predict a qualitative response
Prediction is the average value in each leaf	Prediction is the most common label in each leaf
Splits are chosen to minimize MSE	Splits are chosen to minimize gini index (usually)

Advantages of the Decision Tree

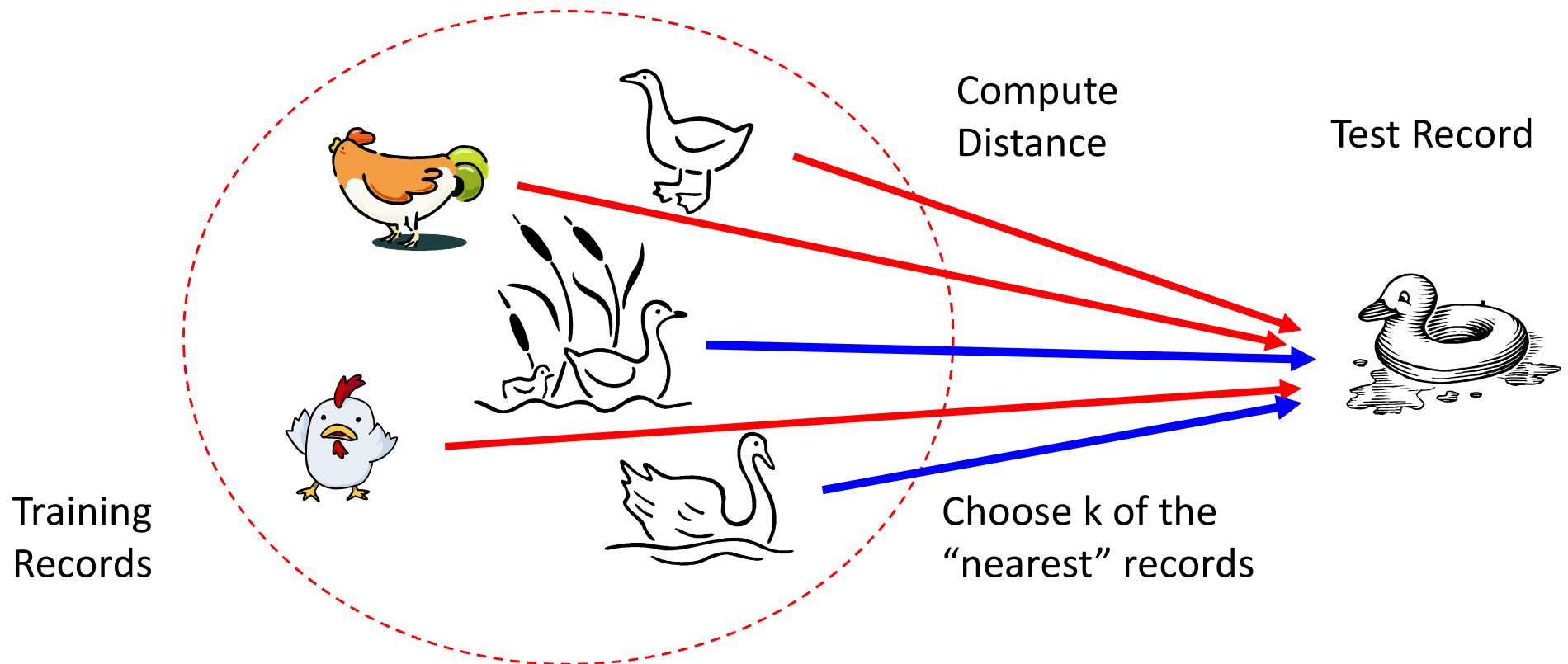
- It is simple to understand as it follows the same process which a human follow while making any decision in real-life.
- It can be very useful for solving decision-related problems.
- It helps to think about all the possible outcomes for a problem.
- There is less requirement of data cleaning compared to other algorithms.

Disadvantages of the Decision Tree

- The decision tree contains lots of layers, which makes it complex.
- It may have an overfitting issue, which can be resolved using the **Random Forest algorithm**.

Nearest Neighbor Classifiers(Knn)

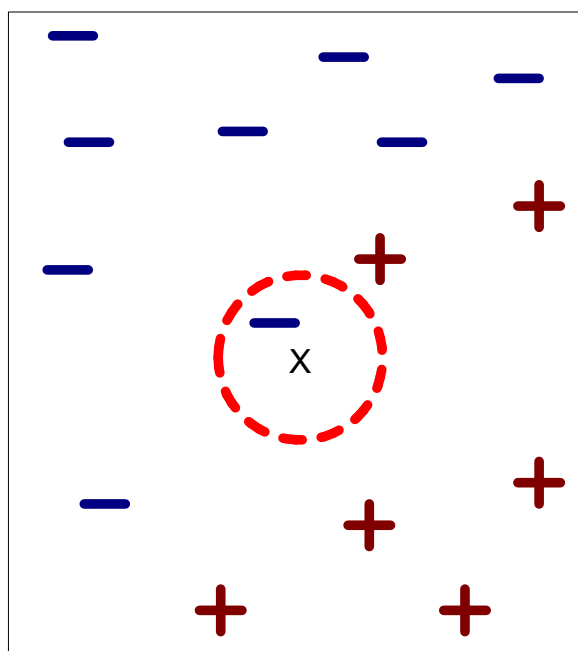
- Basic idea:
 - If it walks like a duck, quacks like a duck, then it's probably a duck



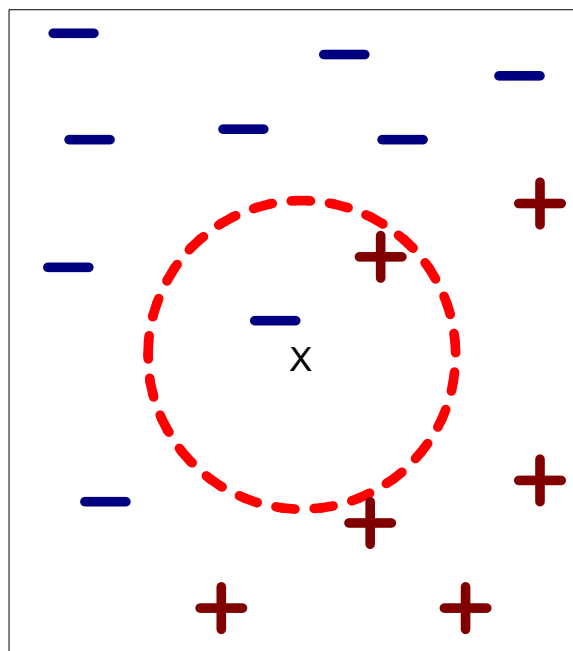
Basic Idea

- **k -NN classification rule** is to assign to a **test** sample the majority category label of its k nearest training samples.
- In practice, k is usually chosen to be odd, so as to avoid ties
- The $k = 1$ rule is generally called the nearest-neighbor classification rule

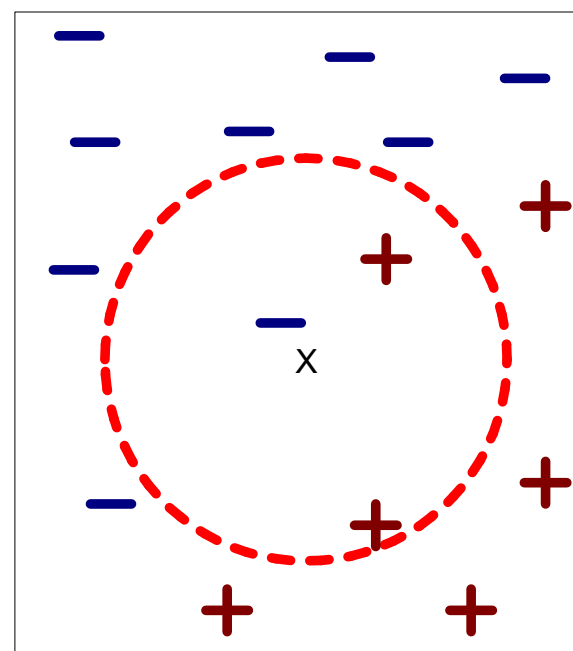
Definition of Nearest Neighbor



(a) 1-nearest neighbor



(b) 2-nearest neighbor



(c) 3-nearest neighbor

K-nearest neighbors of a record x are data points that have the k smallest distance to x

Standardization

- Transform raw feature values into **z-scores**

$$z_{ij} = \frac{x_{ij} - \mu_j}{\sigma_j}$$

—

- x_{ij} is the value for the i^{th} sample and j^{th} feature
 - μ_j is the average of all x_{ij} for feature j
 - σ_j is the standard deviation of all x_{ij} over all input samples
- Range and scale of z-scores should be similar
(providing distributions of raw feature values are alike)

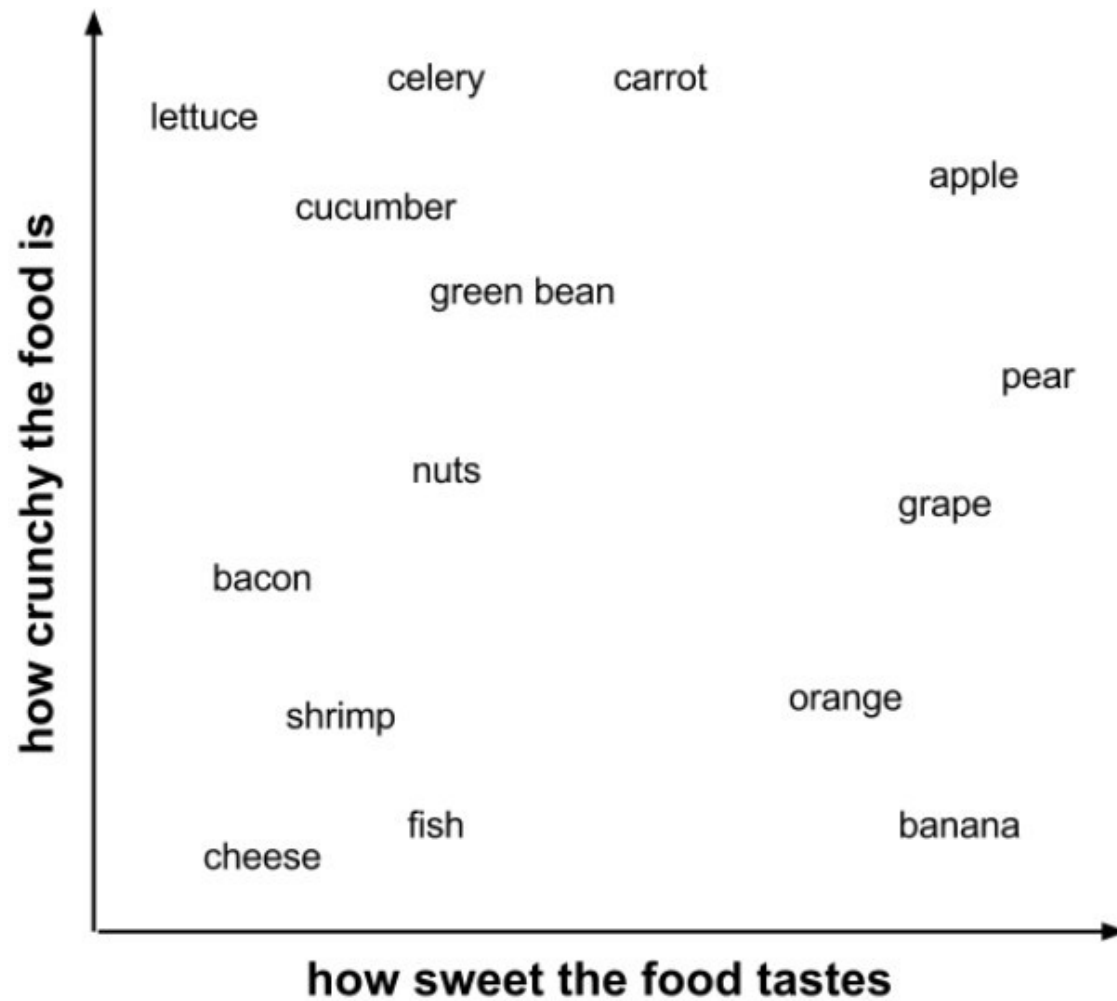
Distance formula

- Euclidian distance: squareroot of sum of squares of differences

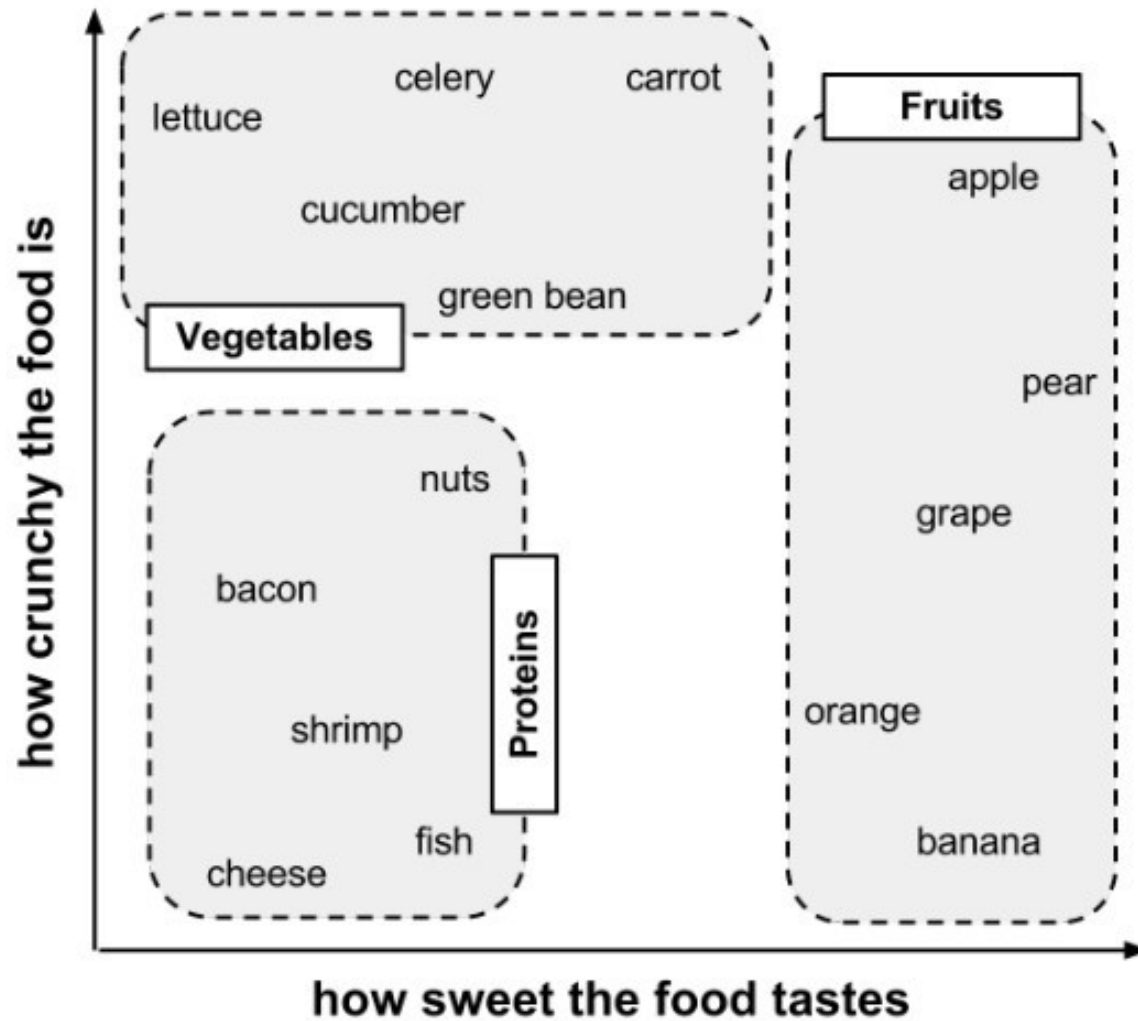
for two features: $\sqrt{(\Delta x)^2 + (\Delta y)^2}$

- Intuition: similar samples should be close to each other
 - May not always apply
(example: quota and actual sales)

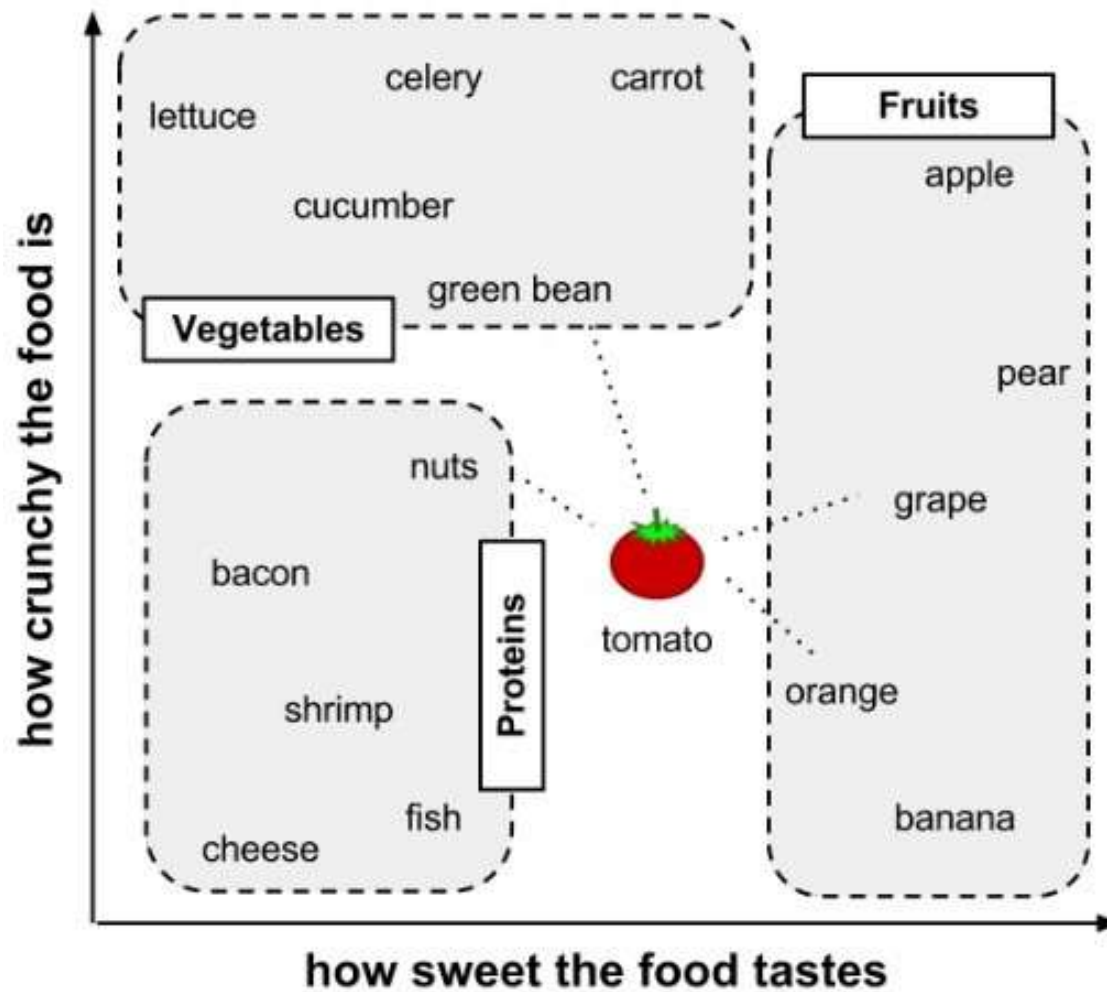
Example:



Example:



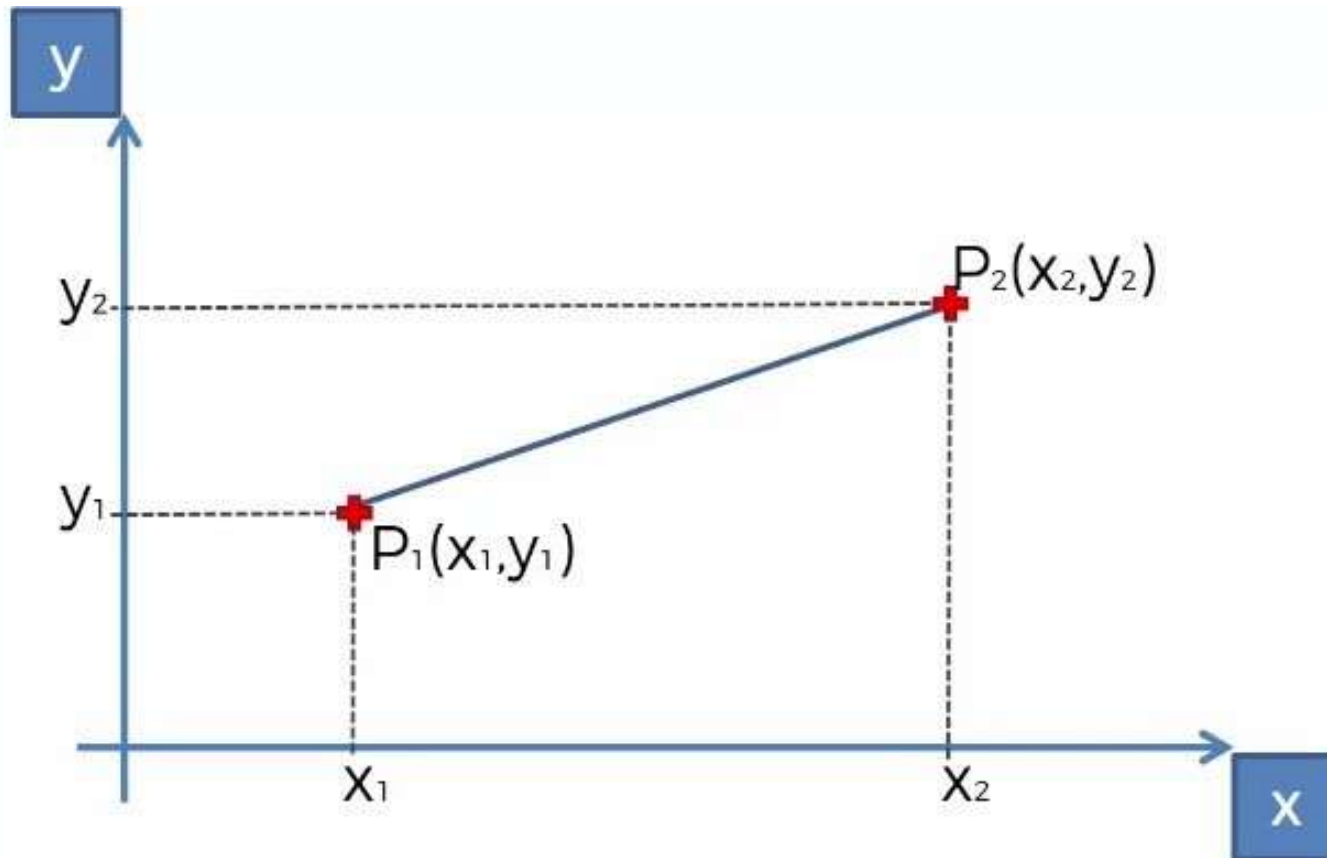
Example:



Calculating Distance

- Locating the tomato's nearest neighbors requires a distance function, or a formula that measures the similarity between two instances.
- There are many different ways to calculate distance.
- Traditionally, the kNN algorithm uses Euclidean distance, which is the distance one would measure if you could use a ruler to connect two points, illustrated in the previous figure by the dotted lines connecting the tomato to its neighbors.

Calculating Distance



Euclidean Distance between P₁ and P₂ = $\sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$

Reference: Super Data Science

Distance

- Euclidean distance is specified by the following formula, where p and q are the examples to be compared, each having n features. The term p_1 refers to the value of the first feature of example p , while q_1 refers to the value of the first feature of example q :

$$\text{dist}(p, q) = \sqrt{(p_1 - q_1)^2 + (p_2 - q_2)^2 + \dots + (p_n - q_n)^2}$$

- The distance formula involves comparing the values of each feature. For example, to calculate the distance between the tomato (sweetness = 6, crunchiness = 4), and the green bean (sweetness = 3, crunchiness = 7), we can use the formula as follows:

$$\text{dist}(\text{tomato}, \text{green bean}) = \sqrt{(6 - 3)^2 + (4 - 7)^2} = 4.2$$

Distance

Distance functions

Euclidean

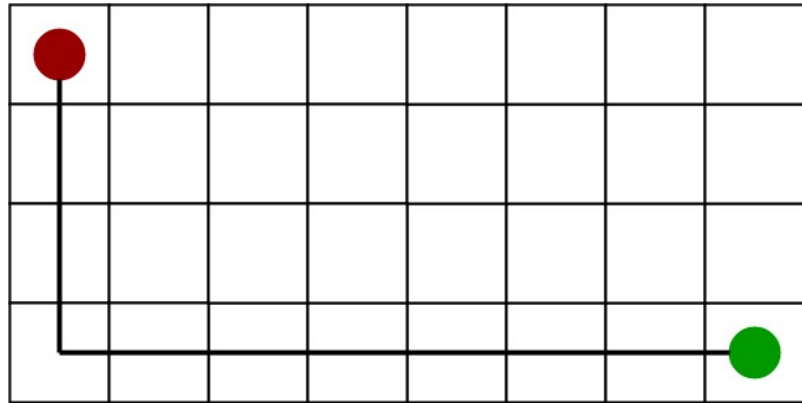
$$\sqrt{\sum_{i=1}^k (x_i - y_i)^2}$$

Manhattan

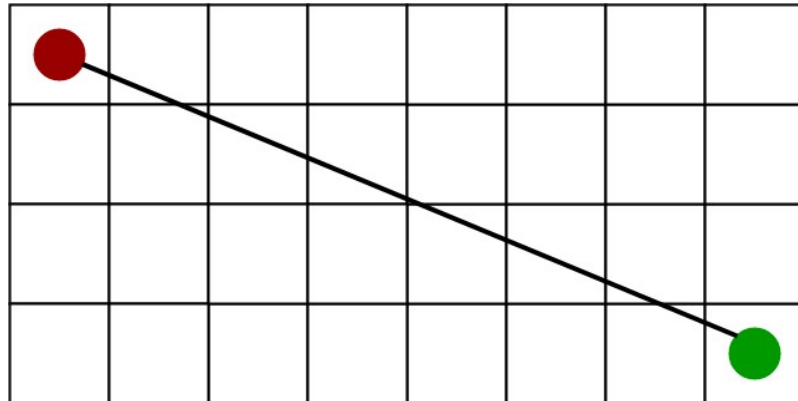
$$\sum_{i=1}^k |x_i - y_i|$$

Distance

Manhattan Distance



Euclidean Distance



Closest Neighbors

ingredient	sweetness	crunchiness	food type	distance to the tomato
grape	8	5	fruit	$\text{sqrt}((6 - 8)^2 + (4 - 5)^2) = 2.2$
green bean	3	7	vegetable	$\text{sqrt}((6 - 3)^2 + (4 - 7)^2) = 4.2$
nuts	3	6	protein	$\text{sqrt}((6 - 3)^2 + (4 - 6)^2) = 3.6$
orange	7	3	fruit	$\text{sqrt}((6 - 7)^2 + (4 - 3)^2) = 1.4$

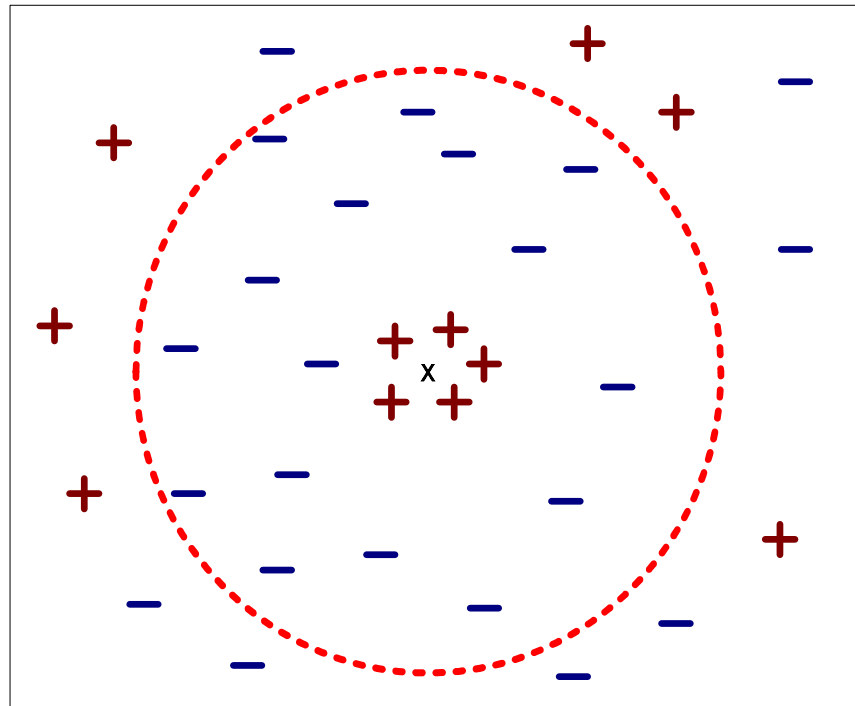
Value of K

- Choosing the value of k:
 - If k is too small, sensitive to noise points
 - If k is too large, neighborhood may include points from other classes

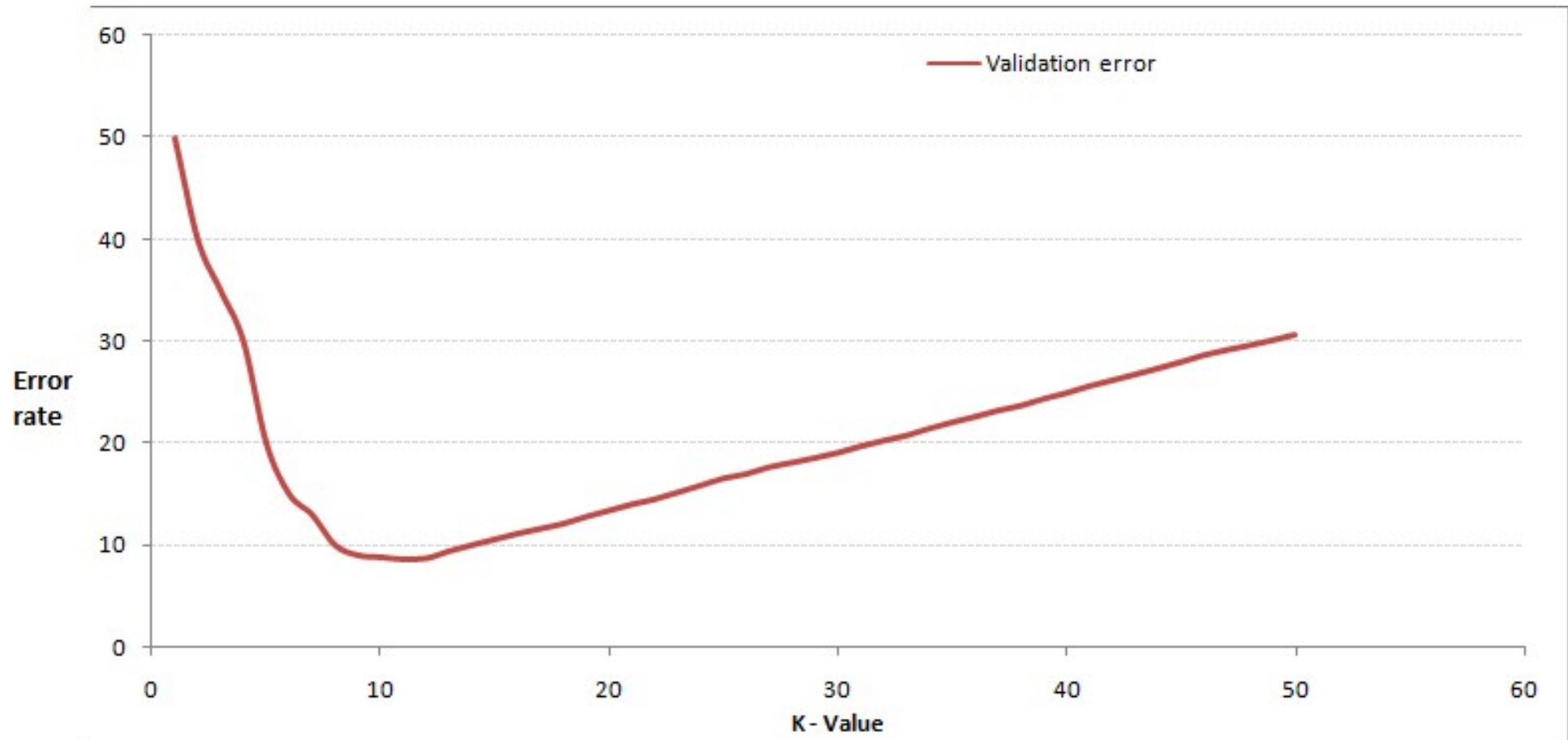
Rule of thumb:

$K = \sqrt{N}$

N: number of training points



Choosing appropriate k



Choosing appropriate k

- In practice, choosing k depends on the **difficulty of the concept to be learned and the number of records in the training data.**
- Typically, k is set somewhere between 3 and 10. One common practice is to set k equal to the square root of the number of training examples.
- *In the classifier, we might set $k = 4$, because there were 15 example ingredients in the training data and the square root of 15 is 3.87.*

Nearest Neighbour : Computational Complexity

- Expensive
 - To determine the nearest neighbour of a query point q , must compute the distance to all N training examples
- Storage Requirements
 - Must store all training data $\mathbf{P}$
- High Dimensional Data
 - “Curse of Dimensionality”
 - Required amount of training data increases exponentially with dimension
 - Computational cost also increases dramatically
 - Partitioning techniques degrade to linear search in high dimension

Knn-Implementation

```
X = [[10,9],[1,4],[10,1],[7,10],[3,10],[1,1]]  
# sweetness and crunchiness  
# apple, bacon, banana, carrot, celery, cheese  
y = ['fruit','protein','fruit','vegetable',  
     'vegetable','protein']
```

```
from sklearn.neighbors import KNeighborsClassifier  
classifier = KNeighborsClassifier(n_neighbors=3)
```

```
classifier.fit(X, y)
```

```
tomato = [[6,4]]  
print classifier.predict(tomato)
```

```
carrot = [[4,9]]  
print classifier.predict(carrot)
```

Unsupervised learning

There are many times when unsupervised learning can be appropriate. Some very common examples include the following:

- When there is **no clear response variable**. *There is nothing that we are* explicitly trying to predict or correlate to other variables.
- To **extract structure** from data where no apparent structure/patterns exist (can be a supervised learning problem).
- When an unsupervised concept called **feature extraction is used**. *Feature extraction is the process of creating new features from existing ones. These new features can be even stronger than the original features.*

Unsupervised learning

There are many times when unsupervised learning can be appropriate. Some very common examples include the following:

- When there is **no clear response variable**. *There is nothing that we are* explicitly trying to predict or correlate to other variables.
- To **extract structure** from data where no apparent structure/patterns exist (can be a supervised learning problem).
- When an unsupervised concept called **feature extraction is used**. *Feature extraction is the process of creating new features from existing ones. These new features can be even stronger than the original features.*

Unsupervised learning

- The **first tends** to be the most common reason that data scientists choose to use unsupervised learning.
- This case arises frequently when we are working with data and we **are not explicitly trying to predict** any of the columns and we merely wish to find patterns of **similar (and dissimilar) groups** of points.
- The **second option** comes into play even if we are explicitly attempting to use a supervised model to predict a response variable. Sometimes simple EDA might **not produce any clear patterns**.
- The **third common reason** to use unsupervised learning is to **extract new features** from features that already exist.
- *It might* produce features that can be used for **presentation purposes** (marketing or otherwise).

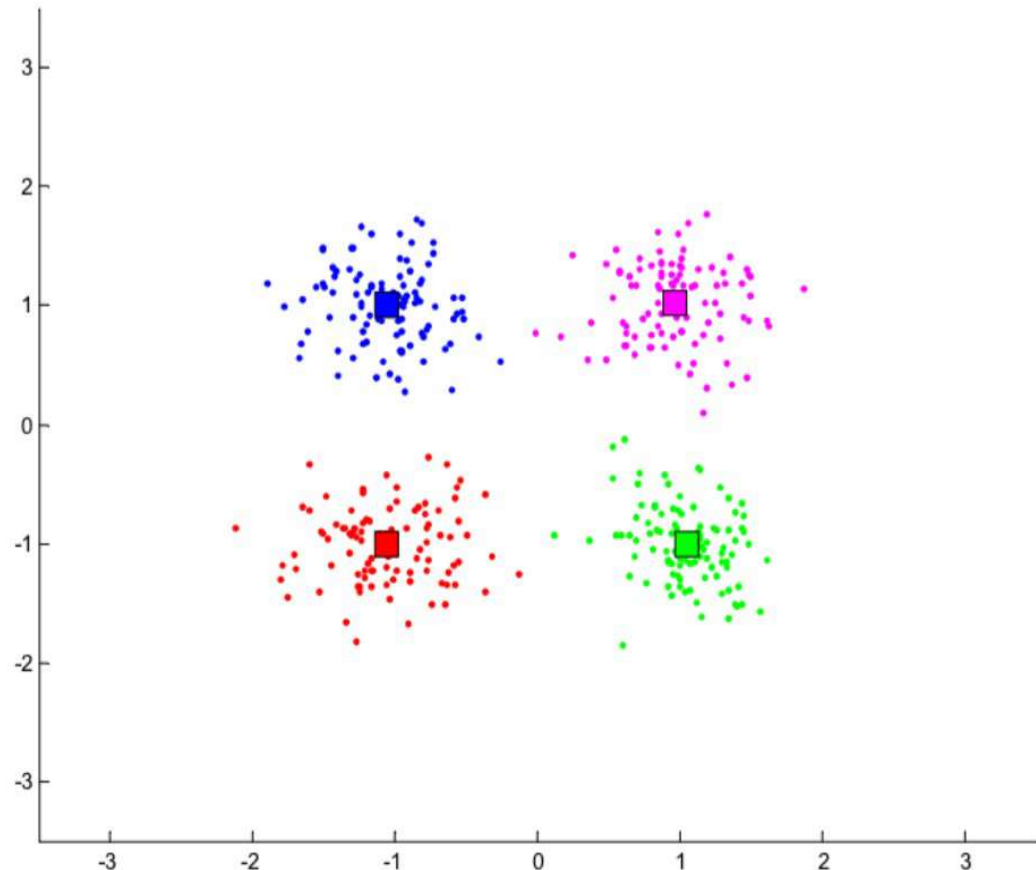
Unsupervised: K-means clustering

- **K-means clustering** is our first example of an unsupervised machine learning model.
- we are **not making predictions**; we are trying instead to extract structure from seemingly **unstructured data**.
- Clustering is a family of unsupervised machine learning models that attempt to group data points into clusters with centroids.
- **Definition:**
Cluster: A *group of data points* that behave **similarly**.
- **Definition:**
Centroid: The *center* of a cluster. Can be thought of as an average point in the cluster.

Unsupervised: K-means clustering

- For example, online shoppers who behave similarly might shop for similar things or at similar shops, whereas similar software companies might make comparable software at comparable prices.

Typical
representation of
K-Means
Clustering. Each
Colour is one Group
(not Class).
Centroid is
presented as Block



Unsupervised: K-means clustering

- In the preceding figure, our human brains can very easily see the difference between the four clusters.
- Namely that the **red cluster** is on the **bottom left of the graph** while the **green cluster** lives in the **bottom right portion** of the graph.
- This means that the red data points are similar to each other, **but *not similar to data points in the other clusters.***
- **centroids of each cluster as the square in each color.**
- Note that *the centroid is not an actual data point, but is merely an abstraction of a cluster* and represents the center of the cluster

Unsupervised: K-means clustering

- In the preceding figure, our human brains can very easily see the difference between the four clusters.
- Namely that the **red cluster** is on the **bottom left of the graph** while the **green cluster** lives in the **bottom right portion** of the graph.
- This means that the red data points are similar to each other, **but *not similar to data points in the other clusters.***
- **centroids of each cluster as the square in each color.**
- Note that *the centroid is not an actual data point, but is merely an abstraction of a cluster* and represents the center of the cluster

Unsupervised: K-means clustering

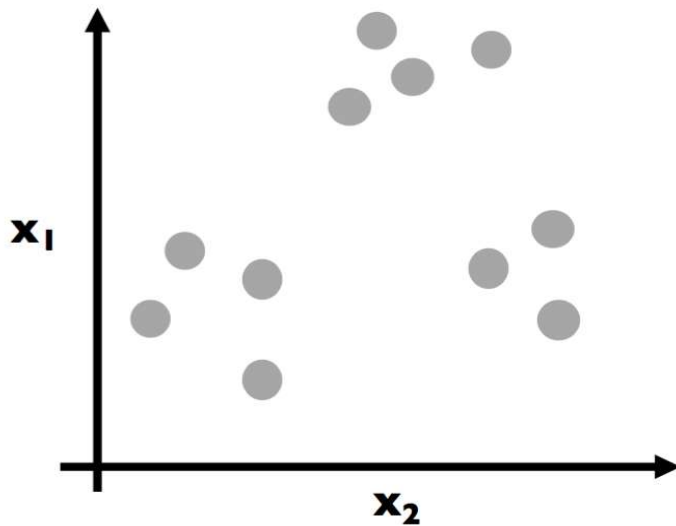
- The concept of *similarity is central to the definition of a cluster, and therefore* to cluster analysis.
- In general, **greater similarity between points leads to better clustering**. In most cases, we turn data into points in ***n -dimensional space and use the distance between these points*** as a form of similarity.
- The centroid of the cluster then is usually **the average of each dimension** (column) for each data point in each cluster.
- So for example, the *centroid of the red cluster is the result of taking the average value of each column of each red data point.*

Unsupervised: K-means clustering

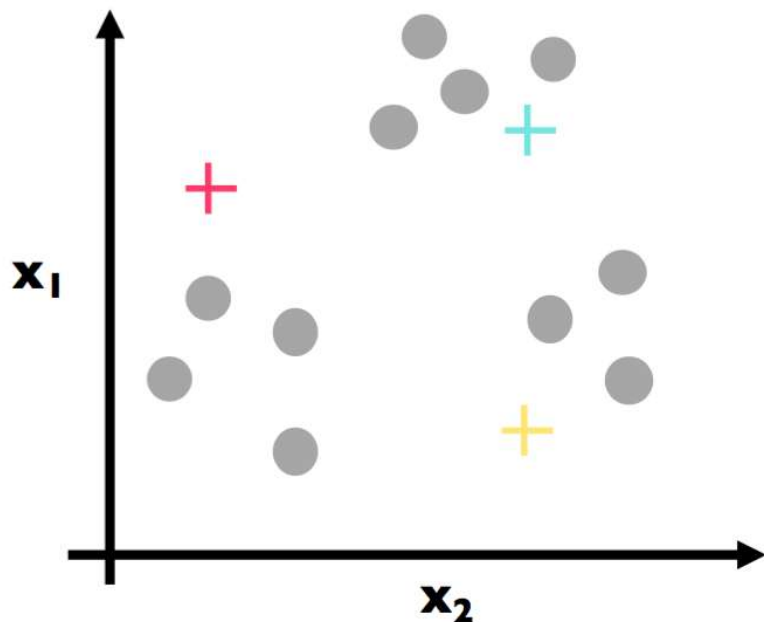
- The goal is to extract and enhance the natural structure of the data. There are many kinds of classification procedures.
- K-means is an iterative method that partitions a data set into k clusters. It works in four steps:
 1. Choose k initial centroids (note that k is an input).
 2. For each point assign the point to the nearest centroid.
 3. Recalculate the centroid positions.
 4. Repeat steps 2-3 until stopping criteria is met.

Illustrative example – data points

Assume Each Dot is a Data Point



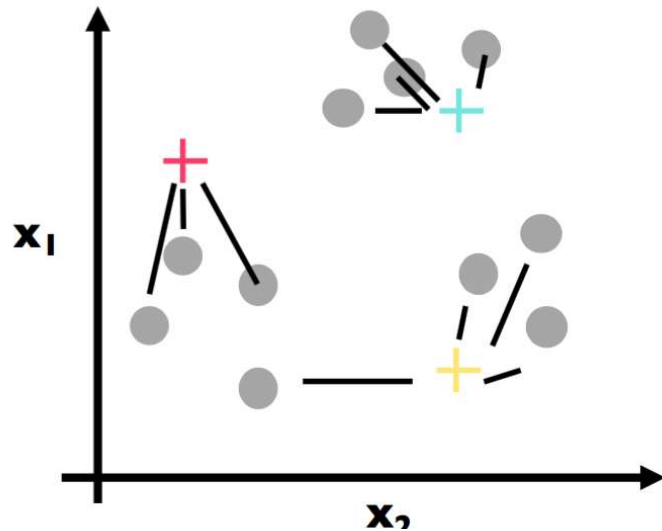
Each dot is colored grey so as to assume no prior grouping before applying the K-means algorithm. The goal here is to eventually color in each dot and create groupings (clusters).



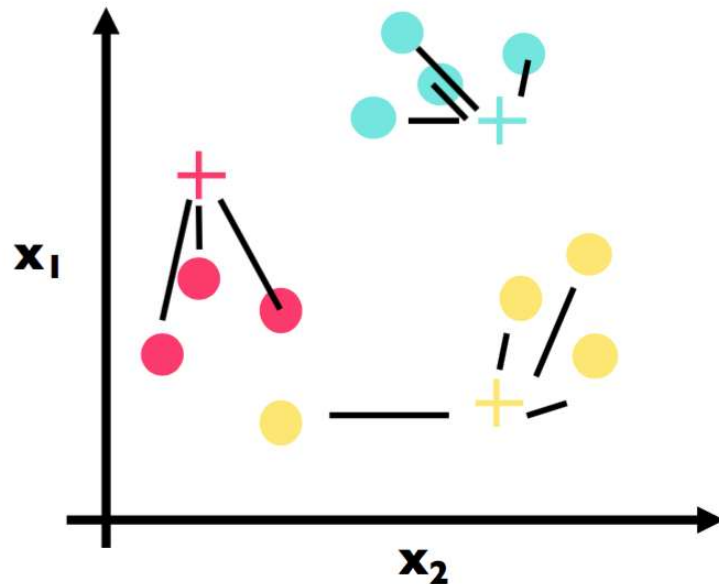
Here, step 1 has been applied. We have (**randomly**) chosen three centroids (red, blue, and yellow).

Illustrative example – data points

Assume Each Dot is a Data Point



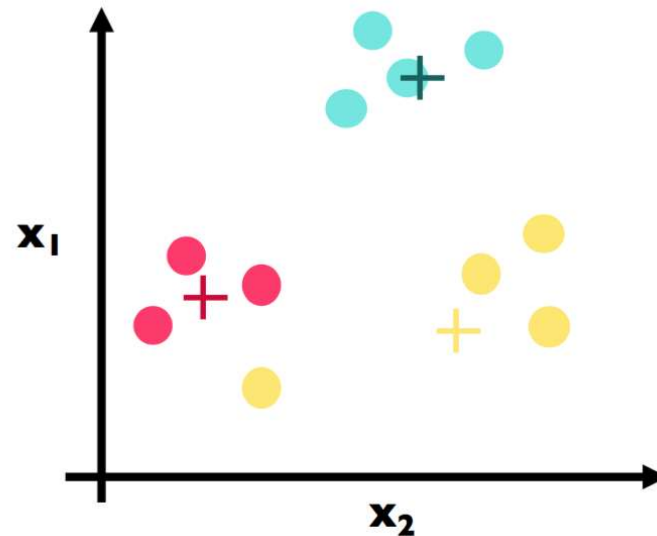
The first part of step 2 has been applied. For each data point, we found the most similar centroid (closest).



The second part of step 2 has been applied here. We have colored in each data point in accordance with its most similar centroid.

Illustrative example – data points

Assume Each Dot is a Data Point



This is step 3 and the crux of K-Means. Note that we have physically moved the centroids to be the actual center of each cluster.

Illustrative example – data points

Assume Each Dot is a Data Point

For example, suppose the three red data points had the coordinates (1, 3), (2, 5), and (3, 4). The center (*red cross*) *would be calculated as follows:*

centroid calculation

```
import numpy as np
```

```
red_point1 = np.array([1, 3])
```

```
red_point2 = np.array([2, 5])
```

```
red_point3 = np.array([3, 4])
```

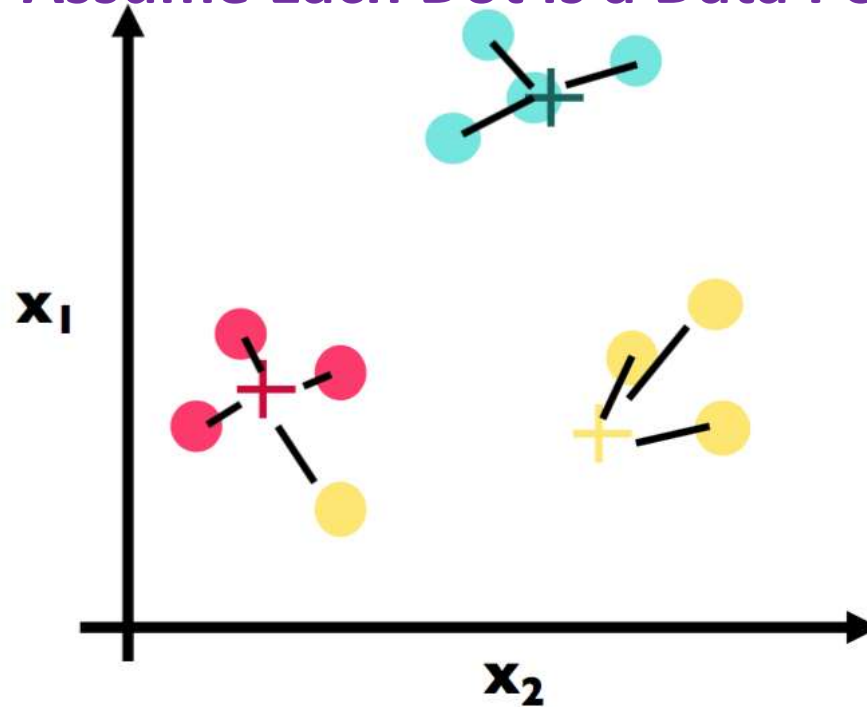
```
red_center = (red_point1 + red_point2 + red_point3) / 3.
```

```
red_center
```

```
# array([ 2.,  4.])
```

Illustrative example – data points

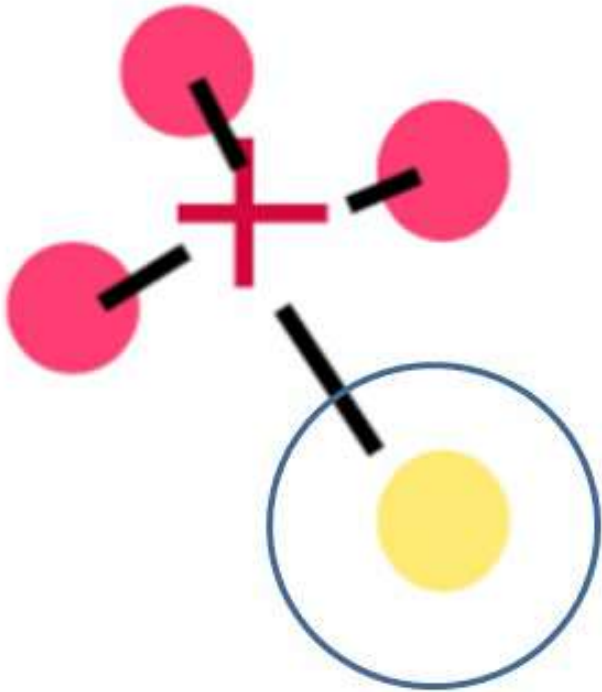
Assume Each Dot is a Data Point



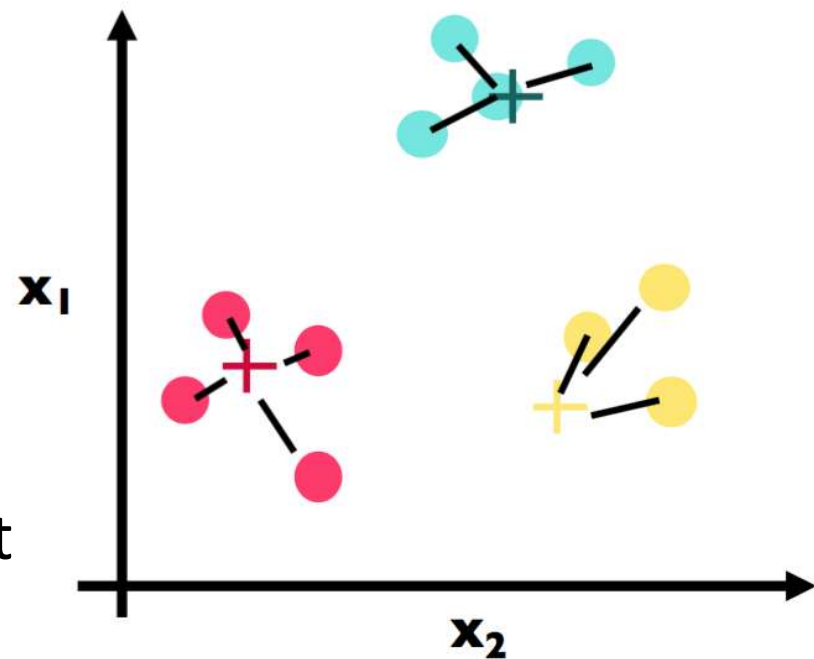
We continue with our algorithm by repeating step 2. Here is the first part where we find the closest center for each point. Note a big change: The point that is circled in the following figure used to be a yellow point, but has changed to be a red cluster point because the yellow cluster moved closer to its yellow constituents.

Illustrative example – data points

Assume Each Dot is a Data Point

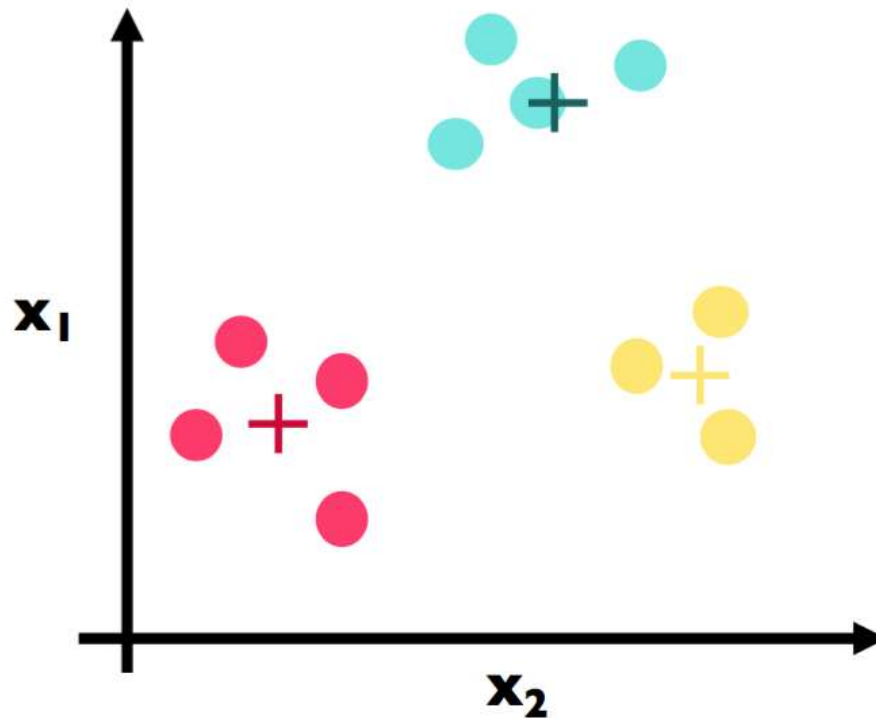


Here is the second part of step 2 again. We have assigned each point to the color of the closest cluster.



Illustrative example – data points

Assume Each Dot is a Data Point



Here Because we have reached a stopping criterion (clusters do not move if we repeat step 2 and 3), we finalize our algorithm and we have our three clusters!

Choosing an optimal number for K and cluster validation

- A big part of K-means clustering is knowing the *optimal number of clusters*. If we knew this number ahead of time, then that might defeat the purpose of even using unsupervised learning. So we need a way to evaluate the output of our cluster analysis.
- The problem here is that because we are *not performing any kind of prediction*, we cannot gauge how *right the algorithm is at predictions*. Metrics such as accuracy and RMSE go right out of the window.

The Silhouette Coefficient

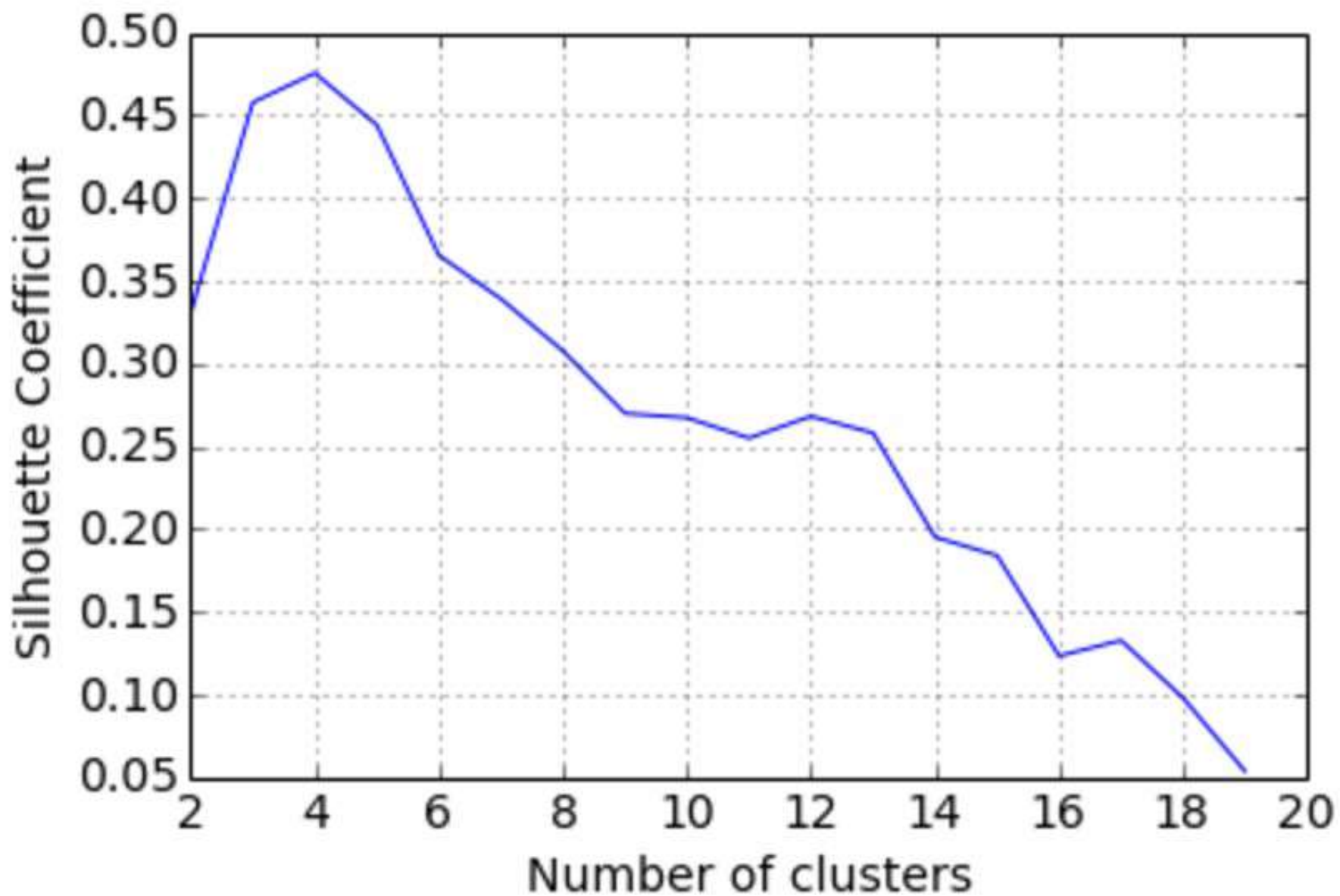
- The **Silhouette Coefficient** is a common metric for evaluating **clustering** performance in situations when the *true cluster assignments are not known*.
- A **Silhouette Coefficient** is calculated for each observation as follows:

$$SC = \frac{b - a}{\max(a, b)}$$

- Let's look a little closer at the specific features of this formula:
- *a: Mean distance to all other points in its cluster*
- *b: Mean distance to all other points in the next nearest cluster*

It ranges from -1 (worst) to 1 (best). A **global score is calculated by taking the mean** score for all observations.

The Silhouette Coefficient



- K-means is a popular algorithm because of its **computational efficiency** and simple and intuitive nature.
- K-means, however, is **highly scale dependent**, and is not suitable for data with widely varying shapes and densities.

K-Means Clustering Example

(2 Iterations with Centroid Updates)

Data Set Used:

Points:

- $P1 = (2, 10)$
- $P2 = (2, 5)$
- $P3 = (8, 4)$
- $P4 = (5, 8)$
- $k = 2$ clusters

Initial Centroids:

- $C1(0) = P1 = (2, 10)$
- $C2(0) = P3 = (8, 4)$

Iteration 1 – Distance Calculations

Assignments:

- $P1 \rightarrow C1$
- $P2 \rightarrow C1$
- $P3 \rightarrow C2$
- $P4 \rightarrow C1$

Clusters:

- Cluster 1: P1, P2, P4
- Cluster 2: P3

Iteration 1 – Updated Centroids

- $C1(1) = \text{Mean}(P1, P2, P4) = (3, 7.67)$
- $C2(1) = P3 = (8, 4)$

Iteration 2 – Assignments

- Using updated centroids:
- $C1 = (3, 7.67)$
- $C2 = (8, 4)$
- Assignments remain:
- Cluster 1: P1, P2, P4
- Cluster 2: P3

Iteration 2 – Centroid Stability

- Centroids do not change:
- $C1 = (3, 7.67)$
- $C2 = (8, 4)$
- → Algorithm converges.

Association Rule Mining

- Association rules are a fundamental concept used to find relationships, correlations or patterns within large sets of data items.
- They describe how often itemsets occur together in transactions and express implications of the form:
- $X \rightarrow Y$
- Where X and Y are disjoint sets of items. This rule suggests that when items in X appear, items in Y tend to appear as well.

- Association rules originated from **market basket analysis** and help retailers and analysts understand customer behavior by discovering item associations in transaction data. For example, a rule stating
- $\{\text{Bread}, \text{Butter}\} \rightarrow \{\text{Milk}\}$
- indicates that customers who buy bread and butter also tend to buy milk.

Key Components

- **Antecedent (X):** The "if" part representing one or more items found in transactions.
- **Consequent (Y):** The "then" part, representing the items likely to be purchased when antecedent items appear.
- Rules are evaluated based on metrics that quantify their strength and usefulness:

Rule Evaluation Metrics

- **1. Support:** Fraction of transactions containing the itemsets in both X and Y.

$$\text{Support}(X \rightarrow Y) = \frac{\text{Number of transactions with } (X \cup Y)}{\text{Total number of transactions}}$$

- Support measures how frequently the combination appears in the data.
- **2. Confidence:** Probability that transactions with X also include Y.

$$\text{Confidence}(X \rightarrow Y) = \frac{\text{Support}(X \cup Y)}{\text{Support}(X)}$$

- Confidence measures the reliability of the inference.

- **3. Lift:** The ratio of observed support to that expected if X and Y were independent.

$$\text{Lift}(X \rightarrow Y) = \frac{\text{Confidence}(X \rightarrow Y)}{\text{Support}(Y)}$$

- $\text{Lift} > 1$ implies a positive association — items occur together more than expected.
- $\text{Lift} = 1$ implies independence.
- $\text{Lift} < 1$ implies a negative association.

Example Transaction Data

Transaction ID	Items
1	Bread, Milk
2	Bread, Diaper, Beer, Eggs
3	Milk, Diaper, Beer, Coke
4	Bread, Milk, Diaper, Beer
5	Bread, Milk, Diaper, Coke

- **Considering the rule:**
- $\{\text{Milk}, \text{Diaper}\} \rightarrow \{\text{Beer}\}$
- **Calculations:**
- $\text{Support}(X \cup Y) = 2/5 = 0.4$
- $\text{Confidence}(X \rightarrow Y) = 2/3 \approx 0.67$
- $\text{Lift}(X \rightarrow Y) = \text{Support}(X \cup Y) / (\text{Support}(X) * \text{Support}(Y)) = 0.4 / (0.6 \times 0.6) = 1.11$ (positive association)

Use Cases

- Let's see the use case of Association rule,
- **Market Basket Analysis:** Identifies products often bought together to improve store layouts and promotions (e.g., bread and butter).
- **Recommendation Systems:** Suggests related items based on buying patterns (e.g., accessories with laptops).
- **Fraud Detection:** Detects unusual transaction patterns indicating fraud.
- **Healthcare Analytics:** Finds links between symptoms, diseases and treatments (e.g., symptom combinations predicting a disease).

Advantages

- **Interpretable and Easy to Explain:** Rules offer clear “if-then” relationships understandable to non-technical stakeholders.
- **Unsupervised Learning:** Works well on unlabeled data to find hidden patterns without prior knowledge.
- **Flexible Data Types:** Effective on transactional, categorical and binary data.
- **Helps in Feature Engineering:** Can be used to create new features for downstream supervised models.

Limitations

- **Large Number of Rules:** Can generate many rules, including trivial or redundant ones, making interpretation hard.
- **Support Threshold Sensitivity:** High support thresholds miss interesting but infrequent patterns; low thresholds generate too many rules.
- **Not Suitable for Continuous Variables:** Requires discretization or binning before use with numerical attributes.
- **Computationally Expensive:** Performance degrades on very large or dense datasets due to combinatorial explosion.
- **Statistical Significance:** High confidence doesn't guarantee a meaningful rule; domain knowledge is essential to validate findings.